Optimizing and Implementing Threshold MAYO

Diego F. Aranha[1], Giacomo Borin[2], Sofía Celi[3] and Guilhem Niot[4]

[1] Aarhus University
[2] IBM Research & University of Zurich
[3] Brave & University of Bristol
[4] PQShield & Univ Rennes, CNRS, IRISA

Abstract. Threshold signatures distribute trust across multiple parties, eliminating single points of failure and reducing insider and key-exfiltration risks—properties that are increasingly important for high-assurance deployments and recently emphasized by NIST's Multi-Party Threshold Cryptography (MPTC) initiative. We present a practical t-out-of-n threshold variant and emulation of MAYO, a post-quantum signature candidate to NIST's call for additional signatures. Our proposal builds upon the threshold MAYO design of Celi, Escudero and Niot (PQCrypto 2025), which we significantly refine to achieve practical performance. To this end, we introduce two algorithmic modifications to MAYO tailored for the distributed setting: (1) **Explicit-Salt MAYO**, which allows for pre-determined salts to enable a single-round online phase; and (2) **Depth-Reduced MAYO**, which restructures the signing algorithm to minimize the depth of secret-dependent operations. We then propose a unified protocol framework that integrate these techniques, plus other MPC specific optimizations, with the goal of minimizing online latency. Finally, we provide a concrete instantiation and local emulation in the dishonest majority setting, secure against active adversaries. Our emulation shows that threshold signing is practical at typical threshold sizes and amenable to deployment. By releasing an open-source implementation and reporting end-to-end performance, this work offers a concrete reference for the thresholdization of post-quantum signatures. Clearly the aforementioned framework is not limited to MAYO, and can be applied to the UOV family of signatures more generally.

Keywords: Threshold Signatures · MPC · multivariate-cryptography

Contents

1 Introduction

Threshold signatures distribute signing power across multiple parties, removing single points of failure and mitigating risks from key exfiltration and insider compromise. These properties are increasingly demanded in real deployments (e.g., HSM-backed services, wallets, and cloud KMS) and research in this field has seen a surge with the recent NIST Multi-Party Threshold Cryptography (MPTC) effort [Nat]. A particularly important requirement is the compatibility of threshold signatures with *single-party* signature standards, to ensure seamless integration into existing systems. While threshold schemes have been extensively studied for classical signatures (e.g., RSA, ECDSA, EdDSA) and are reaching maturity for practical deployment, the landscape for post-quantum (PQ) signatures remains more limited and their practical viability less explored.

Among the NIST selected PQ standards, ML-DSA [LDK+22] stands out as the only scheme with an existing threshold variant that has been implemented and evaluated in realistic settings [BdCE+25, CdPE+26]. Other selected standards (FN-DSA, and SLH-DSA) appear much more challenging to distribute efficiently. In parallel, NIST called for additional signature schemes to expand the diversity of standardized options, with a focus on complementary trade-offs in terms of security, performance, and key/signature sizes. Some of these candidates appear to be more amenable to thresholdization, and exploring their potential is of interest both for the signature and threshold standardization efforts.

One of the candidates is MAYO [Beu22, BCC+23], a multivariate-based signature scheme based on the Oil-Vinegar (OV) paradigm, that stands out due to its compact keys and signatures, fast signing, and relatively conservative security foundation. Celi, Escudero, and Niot [CEN25] initiated the study of threshold MAYO, proposing a t-out-of-n threshold variant, and observed that the rather simple arithmetic structure of OV schemes can be leveraged to design efficient protocols. Interestingly, their techniques are not limited to MAYO, but can be adapted to the well-known `uov` scheme [KPG99] too, also submitted to NIST's call for additional signatures [BCD+24]. However, their work was primarily theoretical, focusing on the protocol design and security analysis, without a concrete instantiation or implementation. In this paper, we extend that line of work on threshold MAYO by refining strictly theoretical protocols into a concrete, practical construction.

Moreover, another important aspect in the design of threshold signatures is the partial non-interactivity of the signing protocol, as achieved, for example, by FROST [KG20] for Schnorr signatures and [EKT24] for lattice-based signatures. In such protocols, the *message-dependent* online signing phase requires only a single round of interaction among the signers, assuming they already performed an efficient offline preprocessing phase. In this work, we also aim to achieve this property for threshold MAYO.

Contributions. We bridge the gap between theory and practice for threshold MAYO by introducing algorithmic refinements, a flexible protocol design, and a concrete protocol emulation. Our contributions directly adapt to UOV, and are as follows:

- **Algorithmic Changes for MPC.** We introduce two modifications to the MAYO signing algorithm to address specific bottlenecks in MPC:
  - **Explicit-Salt MAYO:** A variant where the salt is an input parameter rather than internally sampled. This allows moving salt generation to the offline phase, enabling a single-round online signing phase.
  - **Depth-Reduced MAYO:** An equivalent formulation of the signing logic that enforces oil-space constraints via the system matrix structure rather than secret key multiplication. This minimizes the multiplicative depth of message-dependent operations.

- **Unified Threshold Protocol.** We integrate these techniques into a flexible threshold signing protocol framework. Our design supports configurable trade-offs (via "toggles") between offline complexity and online latency, accommodating different deployment scenarios. We also incorporate various MPC-specific optimizations, such as reorganizing operations to reduce communication rounds and linear system pre-solving.
- **Concrete Instantiation and Emulation.** We provide a concrete instantiation of our protocol in the dishonest majority setting with active security. To validate its performance, we developed a reference emulation in `Go`, which we release as open-source software. This tool simulates the protocol execution in a controlled environment, allowing us to accurately benchmark the computational costs of our proposed variants and demonstrating that threshold MAYO is practical for real-world deployment (focusing on the threshold signing logic and assuming correlated randomness is available).

An especially important optimization we leverage is the *offline/online* paradigm. By shifting all message-independent interaction to a preprocessing phase, we ensure that the online signing phase consists of only a few message-dependent operations. This strategy substantially reduces signing latency. Overall, our results show that threshold MAYO is not only theoretically sound but also practically viable, positioning it as a strong candidate for threshold-friendly post-quantum standardization.

1.1 Related Work

While threshold variants for classical signature schemes are reaching maturity [GHKR08, KG20, CKM23, CGG+20], the post-quantum setting presents new challenges. Early work explored theoretical feasibility [CS19], and have been recently followed by efforts to reach practicality, however focused heavily on lattice-based constructions like threshold variants of ML-DSA [BdCE+25, CdPE+26], and Raccoon [DKM+24, BKL+25]. Comparatively few works introduce isogeny [DM20] or hash-based [KCLM22] schemes. Multivariate cryptography, however, offers a unique advantage due to its simple procedures, a potential confirmed theoretically for MAYO [CEN25] but previously unexploited in practice.

We demonstrate that MAYO and `uov` can achieve competitive performance and we compare our results against recent lattice-based schemes in Table 1. We include results for Threshold Raccoon schemes [DKM+24, BKL+25], and for Threshold ML-DSA [BdCE+25, CdPE+26], representing the state-of-the-art in lattice-based threshold signatures. Our **Explicit-Standard** variant demonstrates sub-millisecond (aggregated) online signing times (0.29 ms and 0.01 ms respectively for (2,2)), outperforming competitors.

Table 1: Online Efficiency Comparison with lattice-based threshold schemes at NIST security level I. †Honest-majority scheme; we report results for corruption threshold $t = 1$.

Scheme	Rounds	|pk| (KB)	|Sig| (KB)	(2,2)		(4,4)	
				Time (ms)	Comm./party	Time (ms)	Comm./party
Th. MAYO$_1$ (Ours)	1	1.4	0.4	0.29	0.4	1.20	0.4
Th. `uov`-Is (Ours)	1	237.9	0.1	0.01	0.1	0.09	0.1
ThRaccoon [DKM+24]	2	3.8	12.4	3.58	35.0	8.44	35.0
Ringtail [BKL+25]	1	4.5	13.4	1.56	10.1	3.68	10.1
Th. ML-DSA [CdPE+26]	4	1.3	2.4	2.26	21.0	10.92	84.0
Th. ML-DSA [BdCE+25]†	Large	1.3	2.4	39.3	416.0	–	–

Acknowledgements

The second author is supported by SNSF Consolidator Grant CryptonIs 213766.

2 Preliminaries

Sets, functions, and distributions. For an integer $N > 0$, we note $[N] = \{0, \ldots, N-1\}$. To denote the *assign* operation, we use $y := f(x)$ when f is deterministic and $y \leftarrow f(x)$ when randomized. We use $||$ to denote concatenation and λ as the security parameter. We use the standard Landau notation $O(\cdot)$ for asymptotics.

Finite fields $\mathbb{F}_q$, and vectors over $\mathbb{F}_q$. Given q, a power of a prime number, we denote $\mathbb{F}_q$ a finite field with q elements. We denote the addition and multiplication of field elements a and b as $a + b$ and ab respectively, and we denote the multiplicative inverse of a as a^{-1}. We denote by $\mathbb{F}_q^n$ the set of column vectors of length n over $\mathbb{F}_q$. Vectors are noted with small bold letters (i.e, $\mathbf{x}$), and we write $(x_0, ..., x_{n-1})$ for the coordinates of $\mathbf{x} \in \mathbb{F}_q^n$.

Matrices. We denote by $\mathbb{F}_q^{m \times n}$ the set of (zero-indexed) matrices over $\mathbb{F}_q$ with m rows and n columns. We denote by $\mathbf{I}_n \in \mathbb{F}_q^{n \times n}$ the identity matrix of size n-by-n. Abusing notation, we see a vector $\mathbf{b} \in \mathbb{F}_q^n$ as a matrix of dimension $n \times 1$. If $\mathbf{A} \in \mathbb{F}_q^{m \times n}$, we denote by $\mathbf{A}[i, j]$ the entry in the i-th row and the j-th column of $\mathbf{A}$, and the j-th column of $\mathbf{A}$ by $\mathbf{A}[:, j] \in \mathbb{F}_q^m$. We say that a matrix $\mathbf{A} \in \mathbb{F}_q^{n \times n}$ is upper triangular if $\mathbf{A}[i, j] = 0$ for all $0 \leq j < i < n$. If $\mathbf{A}, \mathbf{B} \in \mathbb{F}_q^{m \times n}$ are matrices of same size, then we denote their (entry-wise) sum as $\mathbf{A} + \mathbf{B}$. If $\mathbf{A} \in \mathbb{F}_q^{m \times n}$ and $\mathbf{B} \in \mathbb{F}_q^{n \times k}$, then we denote the matrix product by $\mathbf{AB}$, i.e. $\mathbf{AB} \in \mathbb{F}_q^{n \times k}$ is the matrix whose entry in row i and column j is equal to $\sum_{l=0}^{n} \mathbf{A}[i, l]\mathbf{B}[l, j]$. We denote by $\mathbf{A}^\top$ the transpose of $\mathbf{A}$, i.e. the matrix in $\mathbb{F}_q^{n \times m}$ such that $\mathbf{A}^\top[i, j] = \mathbf{A}[j, i]$ for all $0 \leq j < m$ and $0 \leq i < n$. We define the function $\mathsf{Upper} : \mathbb{F}_q^{n \times n} \to \mathbb{F}_q^{n \times n}$ that takes a square matrix $\mathbf{M}$ as input, and outputs the upper triangular matrix $\mathsf{Upper}(\mathbf{M})$, defined as $\mathsf{Upper}(\mathbf{M})_{i,i} = \mathbf{M}_{i,i}$ and $\mathsf{Upper}(\mathbf{M})_{i,j} = \mathbf{M}_{i,j} + \mathbf{M}_{j,i}$ for $i < j$. We also define the function $\mathsf{diag}_k : \mathbb{F}_q^{n \times m} \to \mathbb{F}_q^{kn \times km}$ that takes a matrix $\mathbf{M}$ as input, and outputs the block-diagonal matrix with k copies of $\mathbf{M}$ on the diagonal. To simplify notation, we omit the parameter n, m since they can always be inferred from the context.

2.1 The UC Framework: Model and Functionalities

We use the Universal Composability (UC) framework [Can01] to define and prove security. In this model, we define *ideal functionalities* (denoted $\mathcal{F}$) that act as trusted third parties: they receive inputs, perform computations, and return outputs. These functionalities are instantiated by *protocols* (denoted Π) where parties communicate directly. Security is defined via *simulation*: a protocol Π is secure if the interaction between the parties and the adversary in the *real world* (where Π is executed) can be simulated in the *ideal world* (where only $\mathcal{F}$ exists). Intuitively, this ensures that any information an adversary could learn or any influence they could have in the protocol is already captured by the ideal functionality.

Formally, let $\mathsf{REAL}_{\Pi,\mathcal{A},\mathcal{Z}}(\lambda, z)$ denote the output of the environment $\mathcal{Z}$ in the real-world execution of protocol Π with adversary $\mathcal{A}$, security parameter λ, and auxiliary input z. Similarly, let $\mathsf{IDEAL}_{\mathcal{F},\mathcal{S},\mathcal{Z}}(\lambda, z)$ denote the output of $\mathcal{Z}$ in the ideal-world execution with functionality $\mathcal{F}$ and simulator $\mathcal{S}$.

Definition 1 (UC-security)**.** A protocol Π UC-realizes a functionality $\mathcal{F}$ if for every PPT adversary $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that for every PPT "admissible" environment $\mathcal{Z}$:

$$\{\mathsf{REAL}_{\Pi,\mathcal{A},\mathcal{Z}}(\lambda, z)\}_{\lambda,z} \approx_c \{\mathsf{IDEAL}_{\mathcal{F},\mathcal{S},\mathcal{Z}}(\lambda, z)\}_{\lambda,z}$$

where $\approx_c$ denotes computational indistinguishability.

The $\mathcal{F}$-hybrid Model. We work in the $\mathcal{F}$-hybrid model [CLOS02], where parties run a protocol Π while having access to an ideal functionality $\mathcal{F}$. In this model, parties can exchange messages as in the standard real-world model, but can also invoke $\mathcal{F}$ by sending inputs to it and receiving outputs from it. The UC composition theorem guarantees that if a protocol Π UC-realizes $\mathcal{G}$ in the $\mathcal{F}$-hybrid model, and another protocol P UC-realizes $\mathcal{F}$, then the composed protocol Π^{P} (where calls to $\mathcal{F}$ are replaced by executions of P) UC-realizes $\mathcal{G}$ in the real world.

Implicit Dealing with Aborts and Session IDs. We consider static active adversaries that may cause the protocol to abort. To simplify our descriptions, we assume all functionalities implicitly handle an abort command from the adversary, halting execution and notifying honest parties (*security with abort*). We also omit explicit session identifiers ($\mathsf{sid}, \mathsf{ssid}$), assuming all messages are implicitly associated with the correct session.

Remark 1 (Procedures vs. Protocols). To facilitate modular design, we distinguish between *protocols*, which instantiate specific functionalities, and *procedures*. Procedures are akin to "macros" that outline steps to be executed within a larger protocol but do not themselves define an ideal functionality boundary.

2.2 Core Functionalities and Security Model

We set our work in the *actively secure* model with *dishonest majority* (e.g. up to $t-1$ corrupted parties in a t-out-of-n setting) over a synchronous network [KMOS21, GG19, LNR18], where the adversary can arbitrarily deviate from the protocol and alter messages. In this setting, standard secret-sharing schemes (such as additive or Shamir sharing) do not directly allow to detect miscomputations. To abstractly capture security mechanisms like MACs to detect misbehavior, we work in the *Arithmetic Black Box* (ABB) model [DN03]. The ABB acts as a trusted party performing computations on stored values; a secure execution in the ABB implies a secure real-world execution, provided the underlying instantiation implements operations with active security. Our ABB functionality, $\mathcal{F}_{\mathsf{ABB}}$, supports basic I/O, arithmetic on field elements and matrices, and randomness sampling.

Functionality 1: $\mathcal{F}_{\mathsf{ABB}}$

The functionality maintains a dictionary of stored values $(x, \mathtt{id})$.

- **Input/Output:** On $(\mathsf{input}, \mathtt{id}, P_i)$ from all parties, and $(\mathsf{input}, \mathtt{id}, P_i, x)$ from P_i, stores $(x, \mathtt{id})$. On $(\mathsf{output}, \mathtt{id}, P_i)$, returns x to P_i.
- **Arithmetic:** Computes and stores results for addition, multiplication, and affine combinations (e.g., $ax + b$) on stored values. Supports matrices given compatible dimensions.
- **Randomness:** $(\mathsf{rand}, \mathtt{id})$ stores a fresh secret random value. (coin) outputs a fresh public random value to all parties.

Notation. We omit low-level details such as session IDs and party identifiers in our constructions for simplicity. We denote a value x stored in the ABB as $[\![x]\!]$. We write $[\![z]\!] \leftarrow [\![x]\!] + [\![y]\!]$ for arithmetic operations, and $[\![x]\!] \leftarrow \mathsf{rand}$, $\rho \leftarrow \mathsf{coin}(\{0,1\}^\kappa)$ for randomness. Matrix operations are denoted similarly, e.g., $[\![\mathbf{X}]\!] \leftarrow [\![\mathbf{A}]\!][\![\mathbf{B}]\!]$. This notation is typically reserved for secret-sharing schemes and it is deliberate: in a concrete instantiation of the ABB model, $[\![x]\!]$ would mean that x is secret-shared among the parties. Furthermore, it will not be uncommon that we refer to $[\![x]\!]$ as the parties having "shares" of x.

2.2.1 Solving Linear Systems

As done in the previous work of [CEN25], we consider an additional functionality that allows parties to sample solutions of a linear system of equations. The functionality $\mathcal{F}_{\mathsf{Solve}}$ takes as parameter a distribution $L(r)$ which models leakage as a function of the rank r of $\mathbf{A}$. In the concrete protocol (see Section 4.2), when $\mathbf{A}$ is rank-deficient, the difference $r - r^+$ is revealed (where $r^+ \leftarrow L(r)$) rather than r itself, which potentially provides an extra layer of security. We add one functionality, $\mathsf{solve\text{-}inv}$, that allows parties to also store the inverse of the matrix $\mathbf{A}$ when it is full rank. This does not add complexity to the protocol, since integration is straightforward, but will be relevant for preprocessing

Functionality 2: $\mathcal{F}_{\mathsf{Solve}}$ (L)

The functionality has the *exact same* operations as $\mathcal{F}_{\mathsf{ABB}}$, with the addition that,

- on input $(\mathsf{solve}, \{\mathtt{id}_{ij}\}_{i,j=1,1}^{s,t}, \{\mathtt{id}_i\}_{i=1}^{s}, \{\mathtt{id}'_j\}_{j=1}^{t})$ from the parties, where $s \leq t$, the functionality fetches $\{(a_{ij}, \mathtt{id}_{ij})\}_{i,j=1,1}^{s,t}$ and $\{(b_i, \mathtt{id}_i)\}_{i=1}^{s}$ and proceeds as follows:
  1. Let $\mathbf{A} \in \mathbb{F}_q^{s\times t}$: the (i,j) entry of $\mathbf{A}$ is a_{ij}. Let $\boldsymbol{b} \in \mathbb{F}_q^s$ so that $\boldsymbol{b}[i] = b_i$. Let $r = \mathsf{rank}(\mathbf{A})$.
  2. Sample $r^+ \leftarrow L(r)$. If $r - r^+ < s$, then send $(\mathsf{rank\text{-}defect}, r - r^+)$ to all parties.
  3. Else, sample a uniformly random element $\mathbf{x} \in \mathbb{F}_q^t$ constrained to $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, and store $(\mathbf{x}[j], \mathtt{id}'_j)$ for $j \in [t]$.
- on input $(\mathsf{solve\text{-}inv}, \{\mathtt{id}_{ij}\}_{i,j=1,1}^{s,t}, \{\mathtt{id}_i\}_{i=1}^{s}, \{\mathtt{id}'_j\}_{j=1}^{t}, \{\mathtt{id}'_{i,j}\}_{i,j=1,1}^{t,s})$ from the parties, the functionality call $(\mathsf{solve}, \{\mathtt{id}_{ij}\}_{i,j=1,1}^{s,t}, \{\mathtt{id}_i\}_{i=1}^{s}, \{\mathtt{id}'_j\}_{j=1}^{t})$, and then:
  1. If solve returns $(\mathsf{rank\text{-}defect}, d)$, then send $(\mathsf{rank\text{-}defect}, d)$ to all parties.
  2. Else, let $\mathbf{A} \in \mathbb{F}_q^{s\times t}$ as in solve, sample a matrix $\boldsymbol{X} \in \mathbb{F}_q^{t\times s}$ uniformly at random constrained to $\mathbf{A} \cdot \boldsymbol{X} = \boldsymbol{I}_s$, and store $(\boldsymbol{X}[i,j], \mathtt{id}'_{i,j})$ for $i \in [s]$ and $j \in [t]$.

This functionality can be implemented using different techniques, that may be more or less efficient depending on the protocol we want to instantiate. We discuss how to do that more efficiently than the approach from [CEN25] in Section 4.2.

2.3 Digital Signature Schemes

We start by introducing a formal definition of signature schemes in the standard model. A signature scheme is a tuple of algorithms $(\mathsf{SetUp}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$, defined as follows (henceforth, pp will be implicitly given to all functionality):

$\mathsf{SetUp}(1^\lambda) \to \mathsf{pp}$. Given the security parameter as input 1^λ, the procedure outputs the public parameters pp.

$\mathsf{KeyGen}_{\mathsf{pp}}() \to (\mathsf{pk}, \mathsf{sk})$. The key generation procedure outputs a verification key pk and a private signing key sk.

$\mathsf{Sign}_{\mathsf{pp}}(\mathsf{pk}, \mathsf{sk}, \mathsf{msg}) \to \mathsf{Sig}$. The signing procedure takes as input the public and private keys, as well as a message (msg), and it outputs a signature Sig.

$\mathsf{Verify}_{\mathsf{pp}}(\mathsf{pk}, \mathsf{msg}, \mathsf{Sig}) \to \{0/1\}$. The verification procedure takes as input a verification key pk, a message msg, and a signature Sig, and it outputs 1 if Sig is valid for msg under the verification key pk, and 0 otherwise.

In this work, we ensure the security of our protocols by proving that they UC-realize ideal functionalities that perform the algorithms $(\mathsf{KeyGen}, \mathsf{Sign})$. This is a common way to study the soundness of distributed protocols [DKLs18, CCL+19, ADEO21, CEN25].

We do not pursue the approach defining threshold signatures in the UC framework as in [ADN06, BKP13, BMP22, Mak22].

2.4 MAYO and OV-based schemes

We provide a brief overview of the family Oil-and-Vinegar (OV) of multivariate schemes using the unified description from [CEN25], which relies on the hardness of solving systems of non-linear polynomial equations. We focus in particular on Unbalanced Oil and Vinegar (uov) [KPG99] and MAYO [BCC+23], two promising OV-based signature schemes that have been submitted to the NIST PQC standardization process. uov is well-studied and has small signatures, but relatively large public keys. MAYO reduces key sizes significantly by using structured polynomials and a technique called "whipping".

Overview. The public key is a quadratic map $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$. The secret key is a linear subspace $\mathcal{O} \subset \mathbb{F}_q^n$ (the oil space) of dimension o, such that $\mathcal{P}(\mathbf{o}) = \vec{0}$ for all $\mathbf{o} \in \mathcal{O}$. The oil space is spanned by columns of $[\mathbf{O}^\top \mid \mathbf{I}_o]^\top$, where $\mathbf{O} \in \mathbb{F}_q^{(n-o)\times o}$ is part of the secret key. To sign a message digest $\mathbf{t}$, one looks for $\mathbf{s} = \mathbf{v} + \mathbf{o}$ (where $\mathbf{v}$ is a random "vinegar" vector) satisfying $\mathcal{P}(\mathbf{s}) = \mathbf{t}$. Using the polar form $\mathcal{P}'(\mathbf{x}, \mathbf{y}) = \mathcal{P}(\mathbf{x} + \mathbf{y}) - \mathcal{P}(\mathbf{x}) - \mathcal{P}(\mathbf{y})$, the equation becomes linear in $\mathbf{o}$: $\mathcal{P}'(\mathbf{v}, \mathbf{o}) = \mathbf{t} - \mathcal{P}(\mathbf{v})$. In standard uov, $o = m$, ensuring a solution exists with high probability (approx. $1 - 1/q$). MAYO reduces key sizes by using $o < m$, but this makes the system underdetermined. It solves this via "whipping": the map is lifted to $\mathcal{P}^\star : \mathbb{F}_q^{kn} \to \mathbb{F}_q^m$, taking k input vectors simultaneously to generate enough variables for a solvable system ($ko \geq m$). The map $\mathcal{P}^\star$ is defined for an input $(\mathbf{x}_1, \ldots, \mathbf{x}_k) \in (\mathbb{F}_q^n)^k$ using public fixed matrices $\mathbf{E}_{ij} \in \mathbb{F}_q^{m\times m}$ as

$$\mathcal{P}^\star(\mathbf{x}_1, \ldots, \mathbf{x}_k) := \sum_{i=1}^{k} \mathbf{E}_{ii}\mathcal{P}(\mathbf{x}_i) + \sum_{1\leq i<j\leq k} \mathbf{E}_{ij}\mathcal{P}'(\mathbf{x}_i, \mathbf{x}_j) \tag{1}$$

Unified Description. Our schemes are parameterized by (q, m, n, o, k) – number of polynomials m, number of variables n, oil dimension o, and whipping parameter k (with $k < n - o$) – and fixed public matrices $\mathbf{E}_{i,j} \in \mathbb{F}_q^{m\times m}$. uov uses $k = 1, o = m$; MAYO uses $k > 1$. Key generation (Fig. 1) samples a random oil space $\mathcal{O}$ and m quadratic polynomials p_i that vanish on $\mathcal{O}$. Each P_i is structured as $\begin{pmatrix} \mathbf{P}_i^{(1)} & \mathbf{P}_i^{(2)} \\ \mathbf{0} & \mathbf{P}_i^{(3)} \end{pmatrix}$, with $\mathbf{P}_i^{(3)}$ derived to ensure the vanishing property. Pre-computed values $\mathbf{L}_i$ speed up signing. Verification checks if $\mathcal{P}^\star(\mathbf{s}) = \mathcal{H}(\mathsf{msg})$.

OV-based.KeyGen()

1 : // Sample $\mathbf{O}$ and $(\mathbf{P}^{(1)}, \mathbf{P}^{(2)})$ randomly
2 : $\mathbf{O} \xleftarrow{\$} \mathbb{F}_q^{(n-o)\times o}$
3 : $\{\mathbf{P}_i^{(1)}, \mathbf{P}_i^{(2)}\}_{i\in[m]} \xleftarrow{\$} (\mathbb{F}_q^{(n-o)\times(n-o)}, \mathbb{F}_q^{(n-o)\times o})$
4 : // Compute $\mathbf{P}_i^{(3)} \in \mathbb{F}_q^{o\times o}$
5 : **for** $i \leftarrow 0$ to $m-1$ **do**
6 : $\quad \mathbf{P}_i^{(3)} \leftarrow \mathsf{Upper}(-\mathbf{O}^\mathsf{T}(\mathbf{P}_i^{(1)}\mathbf{O} - \mathbf{P}_i^{(2)}))$
7 : $\quad \mathbf{L}_i \leftarrow ((\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)\mathsf{T}})\mathbf{O} + \mathbf{P}_i^{(2)})$
8 : **return** $(\mathsf{pk} = (\{\mathbf{P}_i^{(1)}, \mathbf{P}_i^{(2)}, \mathbf{P}_i^{(3)}\}_{i\in[m]}), \mathsf{sk} = (\mathbf{O}, \{\mathbf{L}_i\}_{i\in[m]}))$

OV-based.Verify(pk, msg, Sig)

1 : // Parse public key
2 : $\{\mathbf{P}_i^{(1)}, \mathbf{P}_i^{(2)}, \mathbf{P}_i^{(3)}\}_{i\in[m]} \leftarrow \mathsf{pk}$
3 : // Hash salted message
4 : $\mathbf{t} \leftarrow \mathcal{H}(\mathsf{msg}\|\mathsf{salt})$ // $\mathbf{t} \in \mathbf{F}_q^m$
5 : **for** $i \leftarrow 1$ to m **do**
6 : $\quad \mathbf{Y}_i \leftarrow \mathbf{S}\begin{pmatrix} \mathbf{P}_i^{(1)} & \mathbf{P}_i^{(2)} \\ \mathbf{0} & \mathbf{P}_i^{(3)} \end{pmatrix}\mathbf{S}^\mathsf{T}$
7 : $\mathbf{y} \leftarrow \mathsf{OV\text{-}based.Compute\_y}(\{\mathbf{Y}_i\}_{i\in[m]})$ // $\mathbf{y} = \mathcal{P}^\star(\mathbf{s})$
8 : **return** $\mathbf{y} == \mathbf{t}$ // Accept signature if $\mathbf{y} = \mathbf{t}$

Figure 1: Key generation and verification algorithms for OV-based schemes.

Signing. The single-party signing algorithm is illustrated in Fig. 2 using dashed boxes. In broad terms, the signing algorithm first computes a target value $\mathbf{t} = \mathcal{H}(\mathsf{msg}||\mathsf{salt})$, and then computes a preimage $\mathbf{s} \in \mathbb{F}_q^{kn}$ such that $\mathcal{P}^\star(\mathbf{s}) = \mathbf{t}$. This reduces to solving a linear system $\mathbf{A}\mathbf{x} = \mathbf{y}$, where $\mathbf{A}$ and $\mathbf{y}$ are computed from $\mathbf{v}$ and the long-term key material via the procedures OV-based.Compute_A and OV-based.Compute_y. If $\mathbf{A}$ does not have full rank, the process aborts and a fresh $\mathbf{v}$ is sampled. Note that, in the original construction, precomputed linear forms $\mathbf{L}_i$ are used to implicitly encode the constraints corresponding to the oil space $\mathcal{O}$, thereby reducing the dimension of the system and improving performance. We denote by $\mathsf{SampleSolution}(\mathbf{A}, \mathbf{y})$ the procedure that returns a (random) solution to the system $\mathbf{A}\mathbf{x} = \mathbf{y}$, which can be instantiated using standard Gaussian elimination or precomputed inverses [BCH+23a, BCH+23b].

Verification. The verification algorithm is shown in Fig. 1. Given a signature $\mathbf{s}$ on a message msg (with salt salt), the verifier first recomputes $\mathbf{t} = \mathcal{H}(\mathsf{msg}||\mathsf{salt})$, and then checks that $\mathcal{P}^\star(\mathbf{s}) = \mathbf{t}$. If the equality holds, the signature is accepted; otherwise, it is rejected.

Security. The security of OV-based schemes relies on the hardness of the MQ problem, as finding a valid signature without the secret key requires solving a system of multivariate quadratic equations.

3 Alternative Signature Routines for Threshold MAYO

In this section, we introduce two alternative signing algorithms for MAYO, to allow for different trade-offs in a distributed threshold setting. Our goal will be to arrive to an efficient (in terms of rounds) threshold scheme. Note that, typically, in a threshold signature scheme, we split the signing process into an *offline* phase, which is independent of the message to be signed, and an *online* phase, which depends on the message, and should be as efficient as possible. Recall that the MAYO signing algorithm is presented in Fig. 2 (dashed boxes). It can be decomposed into five sub-procedures: (i) message hashing, (ii) preprocessing of quadratic forms, (iii) message-dependent adjustment, (iv) sampling a solution to the linear system, and (v) signature assembly, illustrated with different colours.

We can make two crucial observations about MAYO's signing algorithm in a distributed setting. First, step (i) requires generating fresh randomness (the salt salt) after the message msg is known. This forces salt generation to occur in the *online* phase and therefore requires an interactive coin-tossing protocol (or similar mechanism) among the signers once msg is revealed, adding to the round complexity of the online phase. Second, this signing algorithm is optimized for local execution, where the dominant constraints are CPU cycles and memory, with the pre-computation of secret matrices $\mathbf{L}_i$. In an MPC setting, however, the relevant cost model is often latency, dictated by the round complexity of the protocol and by the depth of secret-by-secret multiplicative operations.

As a stepping stone towards a more efficient threshold signing protocol, we propose two independent modifications of MAYO's signing algorithm.

1. The first and most crucial algorithm – **Explicit-Salt MAYO signing** –, presented in Section 3.1, is a variant of MAYO where the salt is provided as an explicit input rather than being sampled internally. This allows the salt to be pre-determined (e.g. in an offline phase or deterministically derived), resulting in a partially non-interactive online phase requiring only a single round of communication.

2. The second algorithm – **Depth-reduced MAYO signing**, presented in Section 3.2 – is *equivalent* to the original signing algorithm, but is restructured to minimize the depth of multiplicative operations. Multiplicative depth directly impacts the round

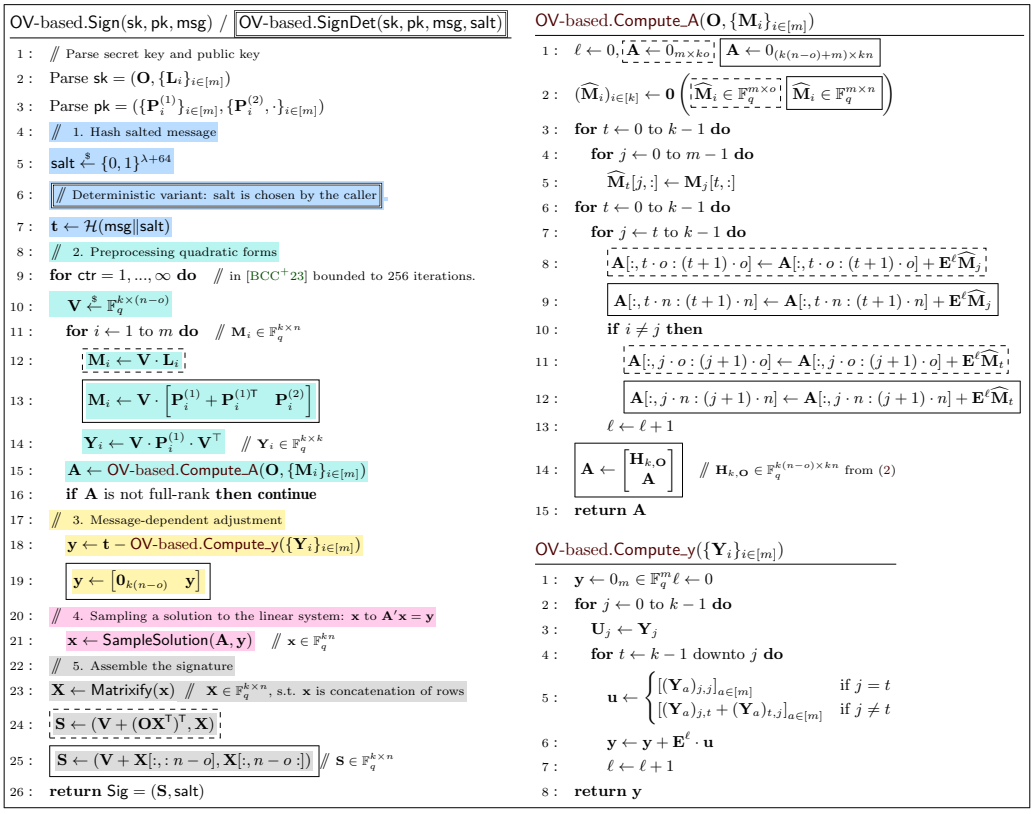

Figure 2: Signing algorithm for MAYO. Dashed boxes show the original operations; solid boxes show the modified operations for multiplicative depth minimisation; double-bordered boxes show the deterministic variant, see Section 3.1.

complexity, which is often the bottleneck in distributed settings where communication latency dominates local computation time.

3.1 Explicit-Salt MAYO

In standard MAYO, the signing algorithm samples a fresh random salt for every signature, which is then hashed with the message to derive the target vector $\mathbf{t} = \mathcal{H}(\mathsf{msg} \,||\, \mathsf{salt})$. In a distributed setting, this internal sampling forces an additional round of interaction (e.g., a coin-tossing protocol) after the message is known. To eliminate this overhead, we introduce a variant called **Explicit-Salt MAYO**, where the salt generation is lifted out of the signing function. Instead of internally sampling a fresh value, the signing algorithm accepts salt as an explicit input parameter. The modified signature generation is then defined as $\sigma \leftarrow \mathsf{SignDet}(\mathsf{sk}, \mathsf{msg}, \mathsf{salt})$, where the target vector is computed as $\mathbf{t} = \mathcal{H}(\mathsf{msg} \,||\, \mathsf{salt})$ using the provided salt.

We show that this modification preserves the original EUF-CMA security of MAYO as long as the tuple (msg, salt) is unique for every signature. Hence, the salt now can be chosen by the caller: it can be (i) pre-agreed during an offline setup phase, (ii) deterministically derived from the message and secret key (e.g., via a PRF), or (iii) externally sampled by a coordinator.

To justify that this modification does not harm security, we briefly revisit the EUF-CMA proof of MAYO. Intuitively, the salt only serves as a source of per-signature randomness in the hash input; allowing the caller (or adversary) to specify it essentially treats the pair

$(\mathsf{msg}, \mathsf{salt})$ as the effective message. To our knowledge, no attack on OV-based signatures would be facilitated by removing the random sampling of the salt, since the target vector $\mathbf{t}$ is still the digest of an hash function modeled as a random oracle. We formalize this in the following proposition.

Proposition 1. *The MAYO signature scheme instantiated with an explicit* salt *is* EUF-CMA *secure in the Random Oracle Model (ROM), assuming that the original scheme [BCC+23, Theorem 1] is* EUF-CMA *secure.*

Formally, let $\mathcal{A}$ be an adversary against the EUF-CMA *security of the explicit salt MAYO scheme, making at most q_H RO queries and q_S signing queries with distinct* $(\mathsf{msg}, \mathsf{salt})$ *pairs. Then, there exists an adversary $\mathcal{B}$ against the* EUF-CMA *security of standard MAYO making at most q_H signing queries, such that*

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\textit{pre-determined-salt}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\textit{standard}}(\mathcal{B}) \cdot 4\,(q_S + 1)$$

Remark 2. Even if no concrete attack is known against the explicit-salt variant, still there is a separation between the security guarantees of the original scheme with internal sampling of the salt and those of the modified scheme. The reasons are twofold:

1. The original security proof of MAYO [BCC+24] incurs a multiplicative loss of $Q_s \cdot B$ (where Q_s is the number of signing queries and B is the restart bound), rendering the reduction vacuous whenever $Q_s \cdot B > 1$. In Proposition 1 the number of signing queries is virtually increased to the number of random oracle calls, thus the security guarantee degrades as the adversary makes more RO queries, potentially biasing the targets vectors $\mathbf{t}$, departing from the guarantees of the original proof. This does not seem to lead to a significant weakening of the security claim in practice as the adversary does not directly observe the signatures defined in the RO simulation, however it departs from the theoretical guarantees of the original scheme.

2. MAYO's specification [BCC+24] notes the salt *somewhat* helps against fault injection or side-channel attacks. These benefits potentially diminished if the salt is pre-computed.

Proof. The proof follows the standard Full-Domain Hash / programming-with-abort argument of Coron [Cor00], specialized to OV-based (MAYO) signing.

Let $\mathcal{A}$ be an adversary against the EUF-CMA security of the pre-determined salt scheme. We construct an adversary $\mathcal{B}$ against the EUF-CMA security of standard MAYO.

Setup. $\mathcal{B}$ receives pk from its challenger and forwards it to $\mathcal{A}$. $\mathcal{B}$ maintains a table L indexed by messages and salt, storing triples $(t_{\mathsf{msg}}, \sigma_{\mathsf{msg}}, b_{\mathsf{msg}})$, where t_{msg} is the value returned to $\mathcal{A}$ as $\mathcal{H}'(\mathsf{msg}||\mathsf{salt})$, σ_{msg} is either a signature $(\mathbf{S}_{\mathsf{msg}}, \mathsf{salt}_{\mathsf{msg}})$ or $\perp$, and $b_{\mathsf{msg}} \in \{0, 1\}$ is a control bit. Let $B \sim \text{Bernoulli}(p)$ for a parameter $p \in (0, 1)$.

Random Oracle $\mathcal{H}'$ simulation. On query $\mathcal{H}'(\mathsf{msg}||\mathsf{salt})$ from $\mathcal{A}$: If $(\mathsf{msg}, \mathsf{salt})$ has a corresponding entry in L, return the stored $\mathbf{t}_{\mathsf{msg}}$; otherwise, sample $b_{\mathsf{msg}} \leftarrow B$, as:

- If $b_{\mathsf{msg}} = 1$, query the *standard* MAYO signing oracle on msg to get $\sigma_{\mathsf{msg}} = (\mathbf{S}_{\mathsf{msg}}, \mathsf{salt}_{\mathsf{msg}})$, then set $\mathbf{t}_{\mathsf{msg}} \leftarrow \mathcal{H}((\mathsf{msg}\|\mathsf{salt})\|\mathsf{salt}_{\mathsf{msg}})$ by querying the challenger-provided RO $\mathcal{H}$. Store $(\mathbf{t}_{\mathsf{msg}}, \sigma_{\mathsf{msg}}, b_{\mathsf{msg}})$ in L and return $\mathbf{t}_{\mathsf{msg}}$.

- If $b_{\mathsf{msg}} = 0$, set $\mathbf{t}_{\mathsf{msg}} \leftarrow \mathcal{H}((\mathsf{msg}\|\mathsf{salt})\|0_{\lambda+64})$ by querying $\mathcal{H}$. Store $(\mathbf{t}_{\mathsf{msg}}, \perp, b_{\mathsf{msg}})$ in L and return $\mathbf{t}_{\mathsf{msg}}$.

Observe that $\mathcal{H}'$ remains correctly distributed, as $\mathcal{H}$ is a RO and pairs $(\mathsf{msg}, \mathsf{salt})$ are queried at most once each.

Signing oracle simulation. On signing query $\mathsf{SignDet}(\mathsf{msg}, \mathsf{salt})$ from $\mathcal{A}$: ensure $(\mathsf{msg}, \mathsf{salt})$ is in L by running the above procedure for $\mathcal{H}'(\mathsf{msg}||\mathsf{salt})$ if needed. If the stored entry has

$\sigma_{\mathsf{msg}} = \perp$, then $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ returns $\mathbf{S}_{\mathsf{msg}}$ to $\mathcal{A}$ (i.e., the pre-determined salt signature component), where $\sigma_{\mathsf{msg}} = (\mathbf{S}_{\mathsf{msg}}, \mathsf{salt}_{\mathsf{msg}})$.

Forgery. When $\mathcal{A}$ outputs a candidate forgery $(\mathsf{msg}^*, \sigma^*)$ where $\sigma^* = (\mathbf{S}^*, \mathsf{salt}^*)$, we assume without loss of generality that $\mathsf{msg}^* \| \mathsf{salt}^*$ was queried to $\mathcal{H}'$, and has a corresponding entry $(\mathbf{t}_{\mathsf{msg}^*}, \sigma_{\mathsf{msg}^*}, b_{\mathsf{msg}^*})$ in L. If $\sigma_{\mathsf{msg}^*} \neq \perp$ then $\mathcal{B}$ aborts. Otherwise ($b_{\mathsf{msg}^*} = 0$), $\mathcal{B}$ outputs the standard-MAYO forgery $((\mathsf{msg}^* \| \mathsf{salt}^*), (\mathbf{S}^*, 0_{\lambda+64}))$.

Analysis. $\mathcal{B}$ aborts if any signing query msg is assigned $b_{\mathsf{msg}} = 0$, or if the forgery message msg^* is assigned $b_{\mathsf{msg}^*} = 1$. Since b_{msg} is sampled independently upon first encounter of each signed message, the probability that $\mathcal{B}$ does not abort is at least $\Pr[\text{no-abort}] \geq p^{q_S}(1-p)$, where p^{q_S} accounts for the event that all q_S signing queries have $b_{\mathsf{msg}} = 1$, and $(1-p)$ accounts for the event that the forgery message has $b_{\mathsf{msg}^*} = 0$.

Conditioned on no-abort, the view of $\mathcal{A}$ is identically distributed to a real execution of the fixed-salt scheme as we always set the RO output of $\mathcal{H}'$ to the output of a fresh query to $\mathcal{H}$, and as we then ensure consistency between signatures and RO outputs. Hence, $\mathcal{A}$ outputs a valid forgery satisfying $P^*(\mathbf{S}^*) = \mathcal{H}((\mathsf{msg}^* \| \mathsf{salt}^*) \| 0_{\lambda+64})$ with probability $\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathrm{fixed-salt}}(\mathcal{A})$, which implies that $(\mathsf{msg}^* \| \mathsf{salt}^*, (\mathbf{S}^*, 0_{\lambda+64}))$ is a valid forgery for standard MAYO. Hence,

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathrm{standard}}(\mathcal{B}) \;\geq\; \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathrm{fixed-salt}}(\mathcal{A}) \cdot p^{q_S}(1-p).$$

To maximize the right-hand side, consider the function $f(p) = p^{q_S}(1-p)$, for $p \in (0,1)$. Its derivative is given by $f'(p) = q_S p^{q_S-1}(1-p) - p^{q_S} = p^{q_S-1}(q_S(1-p) - p)$, which vanishes exactly when $q_S(1-p) = p$, that is, at $p = \frac{q_S}{q_S+1}$. Substituting this value gives $p^{q_S}(1-p) = \left(\frac{q_S}{q_S+1}\right)^{q_S} \cdot \frac{1}{q_S+1} = \left(1 - \frac{1}{q_S+1}\right)^{q_S} \cdot \frac{1}{q_S+1}$, and using the classical bound $(1-\frac{1}{n})^n \geq 1/4$ we conclude that

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathrm{standard}}(\mathcal{B}) \;\geq\; \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathrm{fixed-salt}}(\mathcal{A}) \cdot \left(\frac{q_S}{q_S+1}\right)^{q_S} \cdot \frac{1}{q_S+1} \;\geq\; \frac{\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathrm{fixed-salt}}(\mathcal{A})}{4\,(q_S+1)}.$$

□

3.2 Depth-reduced MAYO signing

The *original* algorithm generates intermediate matrices $\mathbf{M}_i$ by multiplying fresh secret randomness $\mathbf{V}$ with secret components $\mathbf{L}_i$. In MPC, multiplying secret-shared matrices requires interaction, increasing latency. We propose a *depth-reduced* variant that avoids multiplications of secret values. We modify the linear-system construction to enforce oil-space constraints directly within the system matrix, rather than via post-processing with the secret key. This replaces private-by-private multiplications with multiplications by public key components which are essentially free in MPC, at the cost of increasing the linear system's dimension.

Furthermore, the original algorithm multiplies the secret $\mathbf{O}$ with the message-dependent solution $\mathbf{X}$ to form the signature $\mathbf{S}$ (in OV-based.Sign), an operation that must occur online as $\mathbf{X}$ is derived from the message. In our variant, the output is already in the oil space. Therefore, $\mathbf{S}$ is assembled via additions and concatenations of $\mathbf{X}$ and the precomputed $\mathbf{V}$ (line 25), eliminating interactive online operations and reducing MPC round complexity.

This procedure (solid boxes in Figure 2) can be seen as a "naïve" UOV/MAYO formulation: instead of implicitly restricting solutions to the oil space via secret matrices $\mathbf{L}_i$, we construct a system over the full space $\mathbb{F}_q^n$ and *explicitly* enforce constraints. We use the parity-check matrix $\mathbf{H}_{1,\mathbf{O}} = \begin{bmatrix} I_{n-o} & -\mathbf{O} \end{bmatrix}$ (where $\mathbf{z} \in \mathcal{O} \iff \mathbf{H}_{1,\mathbf{O}} \cdot \mathbf{z} = \mathbf{0}$). For MAYO, signatures reside in $\mathcal{O}^k \subseteq \mathbb{F}_q^{kn}$. The corresponding block-diagonal $k(n-o) \times kn$ parity-check matrix

$$\mathbf{H}_{k,\mathbf{O}} = \mathsf{diag}_k\left(\mathbf{H}_{1,\mathbf{O}}\right) \tag{2}$$

satisfies $\mathbf{z} \in \mathcal{O}^k$ if and only if $\mathbf{H}_{k,\mathbf{O}} \cdot \mathbf{z} = \mathbf{0}$.

The algorithm modifies OV-based.Compute_A (green in Figure 2) to explicitly incorporate $\mathbf{H}_{k,\mathbf{O}}$ into $\mathbf{A}$. This constructs a matrix of full dimension n with appended constraint rows. Inputs to OV-based.Compute_A are derived solely from public key components $\mathbf{P}_i^{(1)}, \mathbf{P}_i^{(2)}$ and secret randomness $\mathbf{V}$, eliminating the need for secret matrices $\mathbf{L}_i$. The remaining steps—OV-based.Compute_y, SampleSolution, and OV-based.Verify—remain unchanged.

Even if the modifications in Figure 2 can be regarded as folklore, we provide a proof of its equivalence with the original one for completeness.

Proposition 2. *The modified signing procedure from Figure 2 (solid boxes) outputs signatures that are distributed identically to those produced by the original signing (dashed boxes). Moreover, the rank-deficiency of the matrix* $\mathbf{A}$ *constructed in line 15 of Figure 2 remains the same for any choice of* $\mathbf{V}$.

Proof. We first establish the relation between the matrix $\mathbf{A}$ in the original construction and its modified counterpart $\mathbf{A}'$. Define the matrix $\mathbf{G}_{k,\mathbf{O}} \in \mathbb{F}_q^{kn \times ko}$ generating the oil space $\mathcal{O}^k$:

$$\mathbf{G}_{k,\mathbf{O}} = \mathsf{diag}_k \begin{bmatrix} \mathbf{O} \\ I_o \end{bmatrix} . \tag{3}$$

Observe that $\mathbf{H}_{k,\mathbf{O}} \cdot \mathbf{G}_{k,\mathbf{O}} = \mathbf{0}$, as the rows of $\mathbf{H}_{k,\mathbf{O}}$ are the parity-check equations for $\mathcal{O}^k$. Moreover, by the definition of the secret key, we have

$$\mathbf{L}_i = [\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)^\mathsf{T}} \ \mathbf{P}_i^{(2)}] \cdot \begin{bmatrix} \mathbf{O}^\mathsf{T} & I_o \end{bmatrix}^\mathsf{T} . \tag{4}$$

Let $\mathbf{V} \in \mathbb{F}_q^{k \times (n-o)}$ be the random matrix sampled in line 10 of OV-based.Sign (Figure 2). By (4), the intermediate matrices $\mathbf{M}_i'$ computed in the modified signing algorithm (line 13 of Figure 2) and the matrices $\mathbf{M}_i$ in the original signing algorithm satisfy $\mathbf{M}_i = \mathbf{M}_i' \cdot \begin{bmatrix} \mathbf{O}^\mathsf{T} \mid I_o \end{bmatrix}^\mathsf{T}$. Define $\mathbf{I} = \mathsf{diag}_k \begin{bmatrix} I_{n-o} \\ \mathbf{0}_{o \times (n-o)} \end{bmatrix}$. Applying the above relation to each of the k blocks of $\mathbf{A}$ and $\mathbf{A}'$, and using the parity-check equations, yields

$$\mathbf{A}' \cdot \begin{bmatrix} \mathbf{I} & \mathbf{G}_{k,\mathbf{O}} \end{bmatrix} = \begin{bmatrix} I_{k(n-o)} & \mathbf{0}_{k(n-o) \times ko} \\ * & \mathbf{A} \end{bmatrix} . \tag{5}$$

Since the matrix multiplying $\mathbf{A}'$ in (5) is full rank, the rank deficiency of $\mathbf{A}'$ is identical to that of $\mathbf{A}$, and in particular $\mathsf{rank}(\mathbf{A}') = k(n-o) + \mathsf{rank}(\mathbf{A})$.

It remains to show that the resulting signatures are identically distributed. From (5), $\mathbf{A}'$ has full rank (and thus produces a signature) if and only if $\mathbf{A}$ has full rank. As we are only concerned with executions in which a signature is successfully output, we may condition on both matrices being full rank. Let $\mathcal{S}$ be the solution space of the original linear system defined by $\mathbf{A}$, and let $\mathcal{S}'$ be the solution space corresponding to $\mathbf{A}'$. In the modified algorithm, the signature is computed as $\mathbf{V}$ plus a uniformly sampled element of $\mathcal{S}'$. In the original algorithm, the signature is sampled as a uniformly random element of $\mathcal{S}$, then mapped via $\mathbf{G}_{k,\mathbf{O}}$, and finally $\mathbf{V}$ is added.

It suffices to show that the map $\mathbf{x} \mapsto \mathbf{G}_{k,\mathbf{O}} \cdot \mathbf{x}$ is a bijection between $\mathcal{S}$ and $\mathcal{S}'$, whenever the two matrices $\mathbf{A}$ and $\mathbf{A}'$ are full rank. Since $\mathbf{G}_{k,\mathbf{O}}$ is a full-rank matrix the map is injective. Furthermore, by (5), the image of $\mathcal{S}$ is contained in $\mathcal{S}'$. By the nullity-rank theorem, the dimension (as an affine space) of $\mathcal{S}$ is ko. As it is the same one of $\mathcal{S}'$, the map is surjective. We conclude that the distribution of the signatures is the same. □

Remark 3. Note that for the depth-reduced signing algorithm, the matrices $\mathbf{L}_i$, (obtained in Line 7) are no longer needed, and, hence, they are neither generated nor stored.

4 Practical Threshold MAYO Protocols

In this section, we present a unified framework for threshold MAYO signatures designed to optimize the online message-dependent signing phase. The core idea is to change Line 18 from OV-based.Sign and set $\mathbf{y}$ as $\mathbf{y}_{part} = \text{OV-based.OV-based.Compute\_y}\{\mathbf{Y}_i\}$ and precompute the inverse of the system matrix $\mathbf{A}$ from Line 15. In this way, we can incorporate the target vector $\mathbf{t} = \mathsf{H}(\mathsf{msg}\|\mathsf{salt})$ directly into the solution as $\mathbf{x} = \mathbf{A}^{-1}\mathbf{t} - \mathbf{x}_y$ just before the signature assembly phase (Line 22). In Section 4.3 we detail how to incorporate this intuition into the threshold signing protocol.

Moreover, our framework can be tailored to specific application requirements via two configuration toggles. To achieve this flexibility, our protocol supports modifiable preprocessing and online phases, allowing deployments to trade off between storage size, offline complexity, online round-trips and security guarantees.

Toggle A: Salt Provisioning. On (Explicit Salt): Salt independent of message (based on Section 3.1). Saves one online round: prioritises online performance. **Off (Internal Randomness):** Salt generated after message via coin toss: maximizes provable security, as noted in Remark 2.

Toggle B: Depth-Reduction. On (Depth-Reduced): Larger system ($(k(n-o)+m) \times kn$) removing **one offline round**, using public parity-checks, via Section 3.2. **Off (Standard):** This corresponds to the standard MAYO signing algorithm. Smaller system ($m \times ko$) but **1 extra offline round** to compute $[\![\mathbf{V}]\!] \cdot [\![\mathbf{L}_i]\!]$.

These choices lead to four distinct possible combinations, summarised in Table 2. We then propose concrete protocols that instantiate each profile. For all of them, we propose an optimised procedure for solving systems of linear equations.

Table 2: Protocol Configurations and performance characteristics. For all, we use our optimized linear system solver from Section 4.2, and the trick from Remark 6.

	Protocol		Offline Rounds	Online Rounds	System Size
	[CEN25]		–	$6 + 3/p_{\mathsf{inv}}$	$m \times ko$
	Toggle A	**Toggle B**			
This Work	Randomized	Standard	$1 + 4/p_{\mathsf{inv}}$	2	$m \times ko$
This Work	Randomized	Reduced	$1 + 3/p_{\mathsf{inv}}$	2	$(k(n-o)+m) \times kn$
This Work	Explicit	Standard	$1 + 4/p_{\mathsf{inv}}$	1	$m \times ko$
This Work	Explicit	Reduced	$1 + 3/p_{\mathsf{inv}}$	1	$(k(n-o)+m) \times kn$

4.1 Unified Threshold Functionality

To support these configurations, we define a threshold signing functionality $\mathcal{F}_{\mathsf{ThrSign}}$ that explicitly separates the offline preprocessing phase from the message-dependent online phase. The behavior of the signing query adapts based on the toggles. The preprocessing phase is triggered by the input sign-open, which prepares the necessary components for signing. Once preprocessing is complete, the functionality awaits the input (sign-close, msg), which provides the message, and then produces the signature.

When the randomness is internally sampled (**Toggle A is OFF**), the functionality computes the signature using the standard signing algorithm OV-based.Sign. When an explicit salt is provided (**Toggle A is ON**), it uses the deterministic signing algorithm OV-based.SignDet, with a salt sampled before the message is known. In the latter case, we ensure in the functionality that the same (msg, salt) pair is not signed multiple times as Proposition 1 requires.

Additionally, the functionality incorporates leakage modeling to account for the behavior of the linear system solver used during signing (see Section 4.2). In a local, single-party signing execution, the signer computes the matrix $\mathbf{A}$ (see Fig. 2) and resamples the randomness $\mathbf{V}$ until $\mathbf{A}$ is full rank, allowing a solution to be found. In an MPC context, however, the runtime of the protocol is observable to the adversary; consequently, at a minimum, the protocol leaks the number of attempts required before $\mathbf{A}$ becomes full rank and a solution can be found. Practically, it appears difficult to even limit the leakage to just the number of trials, and [CEN25] proposed a protocol to check the rank of $\mathbf{A}$ at each attempt, that further leaks the rank of the sampled matrix $\mathbf{A}$ at each unsuccessful attempt and discusses the implications of this leakage. We adopt the same approach here. For each unsuccessful attempt, we additionally leak a random variable depending on the *rank* of the corresponding matrix $\mathbf{A}$. This leakage behavior is captured via a distribution $\mathcal{L}(\mathsf{sk})$, parametrised by the secret key sk, which outputs a positive integer representing the rank observed when sampling $\mathbf{V}$ and computing $\mathbf{A}$. Because our protocol offloads the linear-system solving step to the $\mathcal{F}_{\mathsf{Solve}}$ functionality, we additionally apply a second distribution L to the output of $\mathcal{L}(\mathsf{sk})$ to model the rank returned by the `solve` command, which may itself produce a non-full-rank result. As a consequence, Leaks in the ideal world samples from the mixed distribution $L(\mathcal{L}(\mathsf{sk}))$.

Functionality 3: $\mathcal{F}_{\mathsf{ThrSign}}(\mathcal{L}, L)$

Setup phase: On input $(\mathsf{sample\text{-}key})$ from all parties (first time only):

1. Run $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{OV\text{-}based.KeyGen}()$.
2. Internally store sk and output pk to all parties.

Signing: On input $\mathsf{sign\text{-}open}$:

1. **Leakage Simulation.** Repeatedly sample $r'_i \leftarrow \mathcal{L}(\mathsf{sk})$ and set $r_i \leftarrow L(r'_i)$ until r_ℓ is such that a solution can be found (i.e., $r_\ell = m$ if **Toggle B OFF**, or $r_\ell = k(n-o)+m$ if **Toggle B ON**).

   Set $\mathsf{Leaks} = [r_1, \ldots, r_{l-1}]$ and send Leaks to all parties.
2. **Signature Generation.**
   - **Toggle A OFF:** Wait for $(\mathsf{sign\text{-}close}, \mathsf{msg})$, then compute $\mathbf{S} \leftarrow \mathsf{OV\text{-}based.Sign}(\mathsf{sk}, \mathsf{pk}, \mathsf{msg})$.
   - **Toggle A ON:** Sample $\mathsf{salt} \leftarrow \{0,1\}^{\lambda+64}$ and send to all. Wait for $(\mathsf{sign\text{-}close}, \mathsf{msg})$. If $(\mathsf{msg}, \mathsf{salt})$ already signed, abort, otherwise compute $\mathbf{S} \leftarrow \mathsf{OV\text{-}based.SignDet}(\mathsf{sk}, \mathsf{pk}, \mathsf{msg}, \mathsf{salt})$.

   (Note: In both cases, if **Toggle B ON**, use the depth-reduced variant from Figure 2.)
3. **Output.** Send $\mathbf{S}$ to all parties.

Note that it is safe to move the leakage step to the preprocessing phase, as the leakage is independent of the message being signed (see Remark 4 below). This adjustment allows to move more work to the offline phase.

Remark 4. The rank of the matrix $\mathbf{A}$ in Fig. 2 is independent of the message being signed. In fact, the matrix $\mathbf{A}$ is constructed solely from the sampled randomness $\mathbf{V}$, the secret key, and the public key components $\{\mathbf{P}_i^{(1)}, \mathbf{P}_i^{(2)}\}_{i\in[m]}$. The message msg only influences the target vector $\mathbf{y}$ (see line 1), which does not affect the rank of $\mathbf{A}$.

We instantiate functionality $\mathcal{F}_{\mathsf{ThrSign}}$ with Protocol Π_{ThrSign} in Section 4.3. Before detailing the full protocol, we will first present an optimized sub-procedure for solving systems of linear equations ($\mathsf{SampleSolution}$): a critical building block for our construction.

4.2 Concrete Procedure for Solving Systems of Linear Equations

In [CEN25], the authors introduced a secure protocol for obliviously solving systems of linear equations. While correct and generic, that construction incurs multiple rounds of online interaction due to several layers of multiplicative depth in its matrix computations. In contrast, our protocol below provides a depth-optimized solution procedure that avoids these additional rounds and is therefore substantially more efficient in the MPC setting.

Our instantiation of $\mathcal{F}_{\mathsf{Solve}}$ is given as Protocol Π_{Solve} below (we highlight the concrete changes in blue). We stress that, in an actual instantiation of $\mathcal{F}_{\mathsf{ABB}}$, additions are *communication-free*, whereas multiplications require interaction: to highlight this distinction, we explicitly write "parties compute locally" for additions and other non-interactive operations (in the abstract ABB model, no local computation is formally defined, but the notation serves to emphasize the communication costs). A direct count of interaction rounds shows that Protocol Π_{Solve} requires four rounds to output $[\![\mathbf{x}]\!]$ and three rounds to output rank-deficient. Let p_{inv} denote the probability that the sampled matrix is full rank. Then the expected round complexity of Π_{Solve} is $1 + \frac{1}{p_{\mathsf{inv}}} \cdot 3$, where the first term corresponds to the successful attempt and the second models expected restarts when a rank-deficient matrix is encountered.

Procedure 1: Π_{Solve}

The protocol is set in the $\mathcal{F}_{\mathsf{ABB}}$-hybrid. All the commands are forwarded directly to $\mathcal{F}_{\mathsf{ABB}}$.

On input $(\mathsf{solve}, [\![\mathbf{A}]\!], [\![\mathbf{b}]\!])$, where $\mathbf{A}$ has dimensions $s \times t$ and $[\![\mathbf{b}]\!]$ has dimension s, the parties proceed as follows:

1. Parties call $[\![\mathbf{R}]\!] \leftarrow \mathsf{rand}(\mathbb{F}^{s\times s})$ and $[\![\mathbf{S}]\!] \leftarrow \mathsf{rand}(\mathbb{F}^{t\times t})$.
2. Parties call $[\![\mathbf{R}\cdot\mathbf{A}]\!] \leftarrow [\![\mathbf{R}]\!]\cdot[\![\mathbf{A}]\!]$.
3. Parties call $[\![\mathbf{T}]\!] \leftarrow [\![\mathbf{R}\cdot\mathbf{A}]\!]\cdot[\![\mathbf{S}]\!]$. ▷ $\mathbf{R}\cdot\mathbf{A}\cdot\mathbf{S}=\mathbf{T}$.
4. Parties call $[\![\mathbf{h}]\!] \leftarrow [\![\mathbf{R}]\!]\cdot[\![\mathbf{b}]\!]$. ▷ $\mathbf{R}\cdot\mathbf{b}=\mathbf{h}$
5. Parties open $\mathbf{T} \leftarrow [\![\mathbf{T}]\!]$. If $\mathsf{rank}(\mathbf{T}) < s$ parties output $(\mathsf{rank\text{-}deficient}, \mathsf{rank}(\mathbf{T}))$.
6. Otherwise, compute locally $\mathbf{T}^{-1} \in \mathbb{F}_q^{t\times s}$ and a basis $\{\boldsymbol{\beta}_i\}_{i=1}^{t-s}$ of $\ker(\mathbf{T})$ (right kernel).
7. Call locally $([\![z_i]\!])_{i=1}^{t-s} \leftarrow \mathsf{rand}(\mathbb{F}^{t-s})$.
8. Compute locally $[\![\mathbf{z}]\!] \leftarrow \sum_i [\![z_i]\!]\cdot\boldsymbol{\beta}_i$. ▷ $\mathbf{z}$ is uniformly random in $\ker(\mathbf{T})$
9. Compute locally $[\![\mathbf{T}^{-1}\cdot\mathbf{h}+\mathbf{z}]\!] \leftarrow \mathbf{T}^{-1}\cdot[\![\mathbf{h}]\!]+[\![\mathbf{z}]\!]$ and $[\![\mathbf{T}^{-1}\cdot\mathbf{R}]\!] \leftarrow \mathbf{T}^{-1}\cdot[\![\mathbf{R}]\!]$.
10. Parties call $[\![\mathbf{x}]\!] \leftarrow [\![\mathbf{S}]\!]\cdot[\![\mathbf{T}^{-1}\cdot\mathbf{h}+\mathbf{z}]\!]$.
11. ▷ only for $\mathsf{solve\text{-}inv}$ ◁ Parties call $[\![\mathbf{A}^{-1}]\!] \leftarrow [\![\mathbf{S}]\!]\cdot[\![\mathbf{T}^{-1}\cdot\mathbf{R}]\!]$.
12. Output $[\![\mathbf{x}]\!]$ and ▷ only for $\mathsf{solve\text{-}inv}$ ◁ $[\![\mathbf{A}^{-1}]\!]$.

Remark 5. The difference between Π_{Solve} and the protocol in [CEN25] is that the latter explicitly computes $\mathbf{A}^{-1}$, whereas our variant computes $[\![\mathbf{A}^{-1}]\!]$ only as a byproduct, if called with the solve-inv flag. Instead, we precompute a share of $\mathbf{h}=\mathbf{R}\cdot\mathbf{b}$ and defer the batched multiplication by $\mathbf{S}$ to the end of the computation. This reordering allows several multiplications to be executed in parallel, reducing the number of rounds after checking the matrix rank from 6 (as in [CEN25]) to 4.

Theorem 1. *Protocol* Π_{Solve} *instantiates functionality* $\mathcal{F}_{\mathsf{Solve}}(L)$ *in the* $\mathcal{F}_{\mathsf{ABB}}$*-hybrid model, where* $L(r) = \left\{ r - \mathsf{rank}\left(\mathbf{R}\cdot\begin{bmatrix}\mathbf{I}_r & \\ \mathbf{0}_{(s-r)\times r} & \mathbf{0}_{s\times(t-r)}\end{bmatrix}\cdot\mathbf{S}\right) \mid \mathbf{R}\leftarrow\mathbb{F}_q^{s\times s}, \mathbf{S}\leftarrow\mathbb{F}_q^{t\times t}\right\}$.

Proof. To prove Theorem 1, we describe a simulator $\mathcal{S}$ that interacts with the ideal functionality $\mathcal{F}_{\mathsf{Solve}}(L)$ and with the adversary, emulating internally the arithmetic and coin-sampling functionalities, and emulating virtual honest parties.

Initially, $\mathcal{S}$ calls $\mathcal{F}_{\mathsf{Solve}}(L)$, receiving either (as output) some sharings $[\![\mathbf{w}]\!]$ of a solution[1], or $(\mathsf{rankdef}, r - r^+)$ with $r - r^+ < s$ and $r^+ \leftarrow L(r)$. $\mathcal{S}$ proceeds by emulating the two calls to rand in Item 1 by distributing random shares: this is possible because the adversary's shares are independent of the underlying secret. $\mathcal{S}$ also emulates the shared multiplications in Item 2, Item 3, and Item 4. To simulate the opening of $\mathbf{T}$ in Step 5, the simulator samples invertible matrices $\mathbf{U}' \leftarrow \mathbb{F}_q^{s\times s}$ and $\mathbf{V}' \leftarrow \mathbb{F}_q^{t\times t}$ uniformly at random and defines $\mathbf{T}' := \mathbf{U}' \cdot \mathbf{J} \cdot \mathbf{V}'$, where $\mathbf{J}$ is the canonical matrix of rank $r - r^+$. The simulator adjusts the honest parties' shares so that reconstruction yields $\mathbf{T}'$. As in the real protocol, if $\mathsf{rank}(\mathbf{T}') < s$ the simulator outputs $(\mathsf{rankdef}, \mathsf{rank}(\mathbf{T}'))$ and terminates. The distribution of $\mathbf{T}'$ is identical to that of $\mathbf{T} = \mathbf{R} \cdot \mathbf{A} \cdot \mathbf{S}$ conditioned on its rank: indeed, for uniform $\mathbf{R}, \mathbf{S}$ and invertible random changes of basis, $\mathbf{T}$ is uniformly distributed over matrices of the same rank. By construction of $L(r)$, the leakage distribution also matches.

Assume now that $\mathsf{rank}(\mathbf{T}') = s$ and that $\mathcal{F}_{\mathsf{Solve}}(L)$ outputs a sharing $[\![\mathbf{w}]\!]$ of a solution. The simulator emulates Steps 7 and 8 by sampling a uniformly random $\mathbf{z} \in \ker(\mathbf{T}')$ and distributing consistent shares. In Step 9, the simulator computes sharings of $\mathbf{T}'^{-1} \cdot \mathbf{h} + \mathbf{z}and\mathbf{T}'^{-1} \cdot \mathbf{R}$, which is well-defined since $\mathbf{T}'$ is invertible. Finally, in Step 10, the simulator sets the output shares to reconstruct to $\mathbf{w}$, which is consistent with the ideal functionality.

Correctness. Let $\mathbf{x}$ be the value reconstructed in the real execution. We have

$$\begin{aligned}\mathbf{x} &= \mathbf{S} \cdot (\mathbf{T}^{-1}\mathbf{h} + \mathbf{z}) \\ &= \mathbf{S} \cdot (\mathbf{T}^{-1}\mathbf{R}\mathbf{b} + \mathbf{z}).\end{aligned}$$

Multiplying by $\mathbf{A}$ yields

$$\begin{aligned}\mathbf{A}\mathbf{x} &= \mathbf{A}\mathbf{S}\mathbf{T}^{-1}\mathbf{R}\mathbf{b} + \mathbf{A}\mathbf{S}\mathbf{z} \\ &= \mathbf{R}^{-1}\mathbf{T}\mathbf{T}^{-1}\mathbf{R}\mathbf{b} + \mathbf{R}^{-1}\mathbf{T}\mathbf{z} \\ &= \mathbf{b},\end{aligned}$$

since $\mathbf{z} \in \ker(\mathbf{T})$. Hence $\mathbf{x}$ is a valid solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$.

Output distribution. Conditioned on $\mathbf{T}$ being full rank, $\mathbf{z}$ is uniformly distributed in $\ker(\mathbf{T})$ and unknown to the adversary. Therefore, $\mathbf{S}\mathbf{z}$ is uniformly distributed in $\ker(\mathbf{A})$. It follows that $\mathbf{x} = \mathbf{S}(\mathbf{T}^{-1}\mathbf{R}\mathbf{b} + \mathbf{z})$ is a uniformly random solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$, matching the ideal functionality.

□

4.3 The Unified Protocol

The protocol Π_{ThrSign} operates in the $\mathcal{F}_{\mathsf{Solve}}$-hybrid model.

Protocol 2: Π_{ThrSign}

Setup: On $(\mathsf{sample\text{-}key})$, execute π_{KeyGen}. Store $[\![\mathsf{sk}]\!]$ (components depend on Toggle B).
Signing:

- On $\mathsf{sign\text{-}open}$, run π_{Preproc} to generate preprocessed tuple $(\Pi, \mathsf{salt}, \dots)$.
- On $(\mathsf{sign\text{-}close}, \mathsf{msg})$, run $\pi_{\mathsf{SignOnline}}$ using $(\Pi, \mathsf{salt}, \mathsf{msg})$.

Preprocessing Phase. The preprocessing phase π_{Preproc} prepares the linear system components. If **Toggle B (Depth-Reduced)** is active, it is able to compute the matrices $[\![\mathbf{M}_i]\!]$ non-interactively. If **Toggle A (Explicit Salt)** is active, it additionally samples the salt to enable a single-round online phase.

[1]We use $\mathbf{w}$ instead of $\mathbf{x}$ to distinguish between the output in the ideal and real worlds.

Procedure 3: π_{Preproc}

Input: Public key and secret key stored in $\mathcal{F}_{\mathsf{Solve}}$: $\mathsf{pk} = (\mathbf{P}_i^{(1)}, \mathbf{P}_i^{(2)}, \mathbf{P}_i^{(3)})_{i\in[m]}, \mathsf{sk}$.

- **Toggle B OFF:** $\mathsf{sk} = (\llbracket \mathbf{O} \rrbracket, \{\llbracket \mathbf{L}_i \rrbracket\})$.
- **Toggle B ON:** $\mathsf{sk} = \llbracket \mathbf{H} \rrbracket$ (derived from $\mathbf{O}$).

Execution:

1. **Vinegar Generation.** Parties call $\llbracket \mathbf{V} \rrbracket \leftarrow \mathsf{rand}(\mathbb{F}_q^{k\times(n-o)})$.
2. **Matrix Generation.**
   - **Toggle B OFF:** Interactively call $\llbracket \mathbf{M}_i \rrbracket \leftarrow \llbracket \mathbf{V} \rrbracket \cdot \llbracket \mathbf{L}_i \rrbracket$. *(Cost: 1 Round).*
   - **Toggle B ON:** Locally compute $\llbracket \mathbf{M}_i \rrbracket \leftarrow [\llbracket \mathbf{V} \rrbracket \cdot (\mathbf{P}_0^{(1)} + \mathbf{P}_0^{(1)^\mathsf{T}}) \mid \llbracket \mathbf{V} \rrbracket \cdot \mathbf{P}_0^{(2)}]$.
3. **System Construction.** Parties locally compute $\llbracket \mathbf{A} \rrbracket \leftarrow \mathsf{OV\text{-}based.Compute\_A}(\llbracket \mathrm{Key} \rrbracket, \{\llbracket \mathbf{M}_i \rrbracket\})$. (Uses $\llbracket \mathbf{O} \rrbracket$ or $\llbracket \mathbf{H} \rrbracket$ depending on Toggle B).
4. **Target Vector Computation.** For $i \in [m]$, call $\llbracket \mathbf{Y}_i \rrbracket \leftarrow \llbracket \mathbf{V} \rrbracket \cdot \mathbf{P}_i^{(1)} \cdot \llbracket \mathbf{V}^\mathsf{T} \rrbracket$.
   Locally compute $\llbracket \mathbf{y}_{part} \rrbracket \leftarrow \mathsf{OV\text{-}based.Compute\_y}(\{\llbracket \mathbf{Y}_i \rrbracket\})$.
   If **Toggle B ON** set $\llbracket \mathbf{y}_{part} \rrbracket \leftarrow [\mathbf{0} \mid \llbracket \mathbf{y}_{part} \rrbracket]$.
5. **Salt Generation.** If **Toggle A ON**, call $\mathsf{salt} \leftarrow \mathsf{coin}(\{0,1\}^{\lambda+64})$.
6. **Inversion.** Call solve-inv on $(\llbracket \mathbf{A} \rrbracket, \llbracket \mathbf{y}_{part} \rrbracket)$. If rank-deficient, restart. Else, store $(\mathsf{salt}, \llbracket \mathbf{A}^{-1} \rrbracket, \llbracket \mathbf{x}_y \rrbracket)$.
7. **Oil Space Adjustment.** If **Toggle B OFF**, define $\llbracket \mathbf{U} \rrbracket = \mathsf{diag}_k(\llbracket \mathbf{O} \rrbracket)$. Compute and store $\llbracket \mathbf{x}'_y \rrbracket \leftarrow \llbracket \mathbf{U} \rrbracket \cdot \llbracket \mathbf{x}_y \rrbracket$ and $\llbracket \mathbf{B} \rrbracket \leftarrow \llbracket \mathbf{U} \rrbracket \cdot \llbracket \mathbf{A}^{-1} \rrbracket$.

Remark 6 (Optimizing multiplication by $\llbracket \mathbf{U} \rrbracket$). Note that the operation performed in Step 7 (**Toggle B OFF**) involves an additional matrix multiplication on the large $k(n-o) \times ko$ block-diagonal matrix $\llbracket \mathbf{U} \rrbracket$ by some shared vectors/matrices with ko rows. Using the block-diagonal structure of $\mathbf{U}$ allows parallelisation into k independent multiplications of $\mathbf{O}$ by the corresponding k row sub-vectors and sub-matrices.

Looking at the procedure Π_{Solve} (called in the online phase), we can reduce the final round complexity by reorganizing some computations. Both $\llbracket \mathbf{x}'_y \rrbracket$ and $\llbracket \mathbf{B} \rrbracket$ (Steps 10 and 11 of Π_{Solve}) are left multiplied by $\llbracket \mathbf{S} \rrbracket$, which allows us to pre-compute $\llbracket \mathbf{S}' \rrbracket \leftarrow \llbracket \mathbf{U} \rrbracket \cdot \llbracket \mathbf{S} \rrbracket$, for example, during Step 2, and then multiply on the left also by $\llbracket \mathbf{S}' \rrbracket$ while performing the multiplications by $\llbracket \mathbf{S} \rrbracket$ in Π_{Solve}. Hence, Π_{Solve} also benefits from pre-processing.

Remark 7 (**Toggle B** on structured multiplications). When **Toggle B** is ON (Depth-Reduced) part of the input $\mathbf{A}$ is structured, since the first $k(n-o)$ columns correspond to the parity-check matrix $\mathbf{H}_{k,\mathbf{O}}$ (2), and the first $k(n-o)$ entries of the input vector share correspond to zeroes (see Line 19 of Figure 2). Thus, not all multiplications by random matrices are necessary: in particular, we can structure $\mathbf{R}, \mathbf{S}$. Before applying Π_{Solve} we can reorganize the matrix $\mathbf{A}'$ in such a way that the first $k(n-o)$ rows are the identity matrix, so that it has the form $\begin{pmatrix} I_{k(n-o)} & \mathsf{diag}_k(\mathbf{O}) \\ \mathbf{A}_1 & \mathbf{A}_2 \end{pmatrix}$, where $\mathbf{A}_1$ is a matrix of dimension $m \times k(n-o)$ and $\mathbf{A}_2$ is a matrix of dimension $m \times ko$. Then, we can modify Π_{Solve} so that we multiply by random matrices $\mathbf{R}' = \begin{pmatrix} I_{k(n-o)} & \mathbf{0} \\ \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix}$ and $\mathbf{S}' = \begin{pmatrix} I_{k(n-o)} & \mathbf{S}_1 \\ \mathbf{0} & \mathbf{S}_2 \end{pmatrix}$. Note that all multiplications by zeros can be trivially skipped.

Corollary 1. *The protocol Π_{Solve}, modified as in Remark 7, with input a $k(n-o)+s' \times k(n-o)+t'$ matrix from Line 3 of π_{Preproc}, instantiates $\mathcal{F}_{\mathsf{Solve}}(L)$ in the $\mathcal{F}_{\mathsf{ABB}}$-hybrid model with L as in (1) for $s = s'$ and $t = t'$.*

Sketch of Proof. Using the $\mathcal{F}_{\mathsf{ABB}}$ functionality, we can reorder the rows of the input matrix as described in Remark 7. Let us denote the blocks of the input matrix as $\llbracket \begin{pmatrix} I_{k(n-o)} & \mathbf{A}_0 \\ \mathbf{A}_1 & \mathbf{A}_2 \end{pmatrix} \rrbracket$.

Using the $\mathcal{F}_{\mathsf{ABB}}$ functionality we can multiply on the left by $\begin{pmatrix} I_{k(n-o)} & \mathbf{0} \\ -[\![\mathbf{A}_1]\!] & I_m \end{pmatrix}$ and on the right by $\begin{pmatrix} I_{k(n-o)} & -[\![\mathbf{A}_0]\!] \\ \mathbf{0} & I_{ko} \end{pmatrix}$ to obtain the matrix $\begin{pmatrix} I_{k(n-o)} & \mathbf{0} \\ \mathbf{0} & [\![\mathbf{A}_2 - \mathbf{A}_1\mathbf{A}_0]\!] \end{pmatrix}$. Clearly, this matrix has the same rank as the input matrix. We can now consider the simulator $\mathcal{S}$ from the proof of Theorem 1 and define a new simulator $\mathcal{S}'$ that performs the previously defined operations, and then simulates the optimized protocol using the same operations as $\mathcal{S}$, with the only exception that, whenever $\mathcal{S}$ would apply some operations using a $c \times c$ operation we augment it to a $(c + k(n-o)) \times (c + k(n-o))$ (for $c \in \{s, t\}$) operation that uses the augmented matrix with the identity on the first $k(n-o)$ rows and columns, and vice versa when extracting the matrix to be used for $\mathcal{S}$. By composing $\mathcal{S}$ operations (sampling, multiplying and opening) with the previous two multiplications, we obtain $\mathcal{S}$'. We can do this as multiplying by these two matrices does not change the rank and gives matrix of the same shapes as $\mathbf{R}'$, $\mathbf{S}'$ from Remark 7.

The correctness and output distribution of the shares of the output vector follows directly by the same arguments as in the proof of Theorem 1. □

Online Signing. The online phase (in $\pi_{\mathsf{SignOnline}}$) consumes the preprocessed values. If **Explicit Salt**, the procedure is single round of reconstruction. If **Internal Randomness**, it requires two rounds: one for the coin-toss and one for reconstruction.

Procedure 4: $\pi_{\mathsf{SignOnline}}$

Input: pk, sk stored in $\mathcal{F}_{\mathsf{Solve}}$, message $\mathsf{msg} \in \{0,1\}^*$.

1. **Salt Establishment.**
   - **Toggle A OFF:** Parties perform coin-toss $\mathsf{salt} \leftarrow \mathsf{coin}(\{0,1\}^{\lambda+64})$.
   - **Toggle A ON:** Fetch salt from preprocessing.
2. **Solution Derivation.**
   - **Toggle B OFF:** Let $\mathbf{t} = \mathsf{H}(\mathsf{msg}\|\mathsf{salt})$. Fetch $([\![\mathbf{A}^{-1}]\!], [\![\mathbf{x}_y]\!], [\![\mathbf{B}]\!], [\![\mathbf{x}'_y]\!])$. Locally compute $[\![\mathbf{x}]\!] \leftarrow [\![\mathbf{A}^{-1}]\!]\mathbf{t} - [\![\mathbf{x}_y]\!]$ and $[\![\mathbf{x}']\!] \leftarrow [\![\mathbf{B}]\!]\mathbf{t} - [\![\mathbf{x}'_y]\!]$.
   - **Toggle B ON:** Let $\mathbf{t} = (0, \ldots, 0, \mathsf{H}(\mathsf{msg}\|\mathsf{salt}))$. Fetch $([\![\mathbf{A}^{-1}]\!], [\![\mathbf{x}_y]\!])$. Locally compute $[\![\mathbf{x}]\!] \leftarrow [\![\mathbf{A}^{-1}]\!]\mathbf{t} - [\![\mathbf{x}_y]\!]$.
3. **Signature Reconstruction.** Compute $[\![\mathbf{X}]\!] \leftarrow \mathsf{Matrixify}([\![\mathbf{x}]\!])$ (and $[\![\mathbf{X}']\!] \leftarrow \mathsf{Matrixify}([\![\mathbf{x}']\!])$ if Toggle B is OFF).
   - **Toggle B OFF:** Locally: $[\![\mathbf{S}]\!] \leftarrow ([\![\mathbf{V}]\!] + [\![\mathbf{X}']\!], [\![\mathbf{X}]\!])$.
   - **Toggle B ON:** Locally: $[\![\mathbf{S}]\!] \leftarrow ([\![\mathbf{V}]\!] + [\![\mathbf{X}]\!][:, : n-o], [\![\mathbf{X}]\!][:, n-o :])$.

   Finally, interactively open $[\![\mathbf{S}]\!]$ to obtain $\mathbf{S}$.

4.4 Security and Trade-offs

Theorem 2. *For a polynomial number of signing queries q_s, the protocol Π_{ThrSign} statistically UC-realizes the functionality $\mathcal{F}_{\mathsf{ThrSign}}(\mathcal{L}, L)$ in the $\mathcal{F}_{\mathsf{Solve}}(L)$-hybrid model.*

Sketch of the proof. We can apply the same proof strategy as [CEN25, Theorem 2], with only one change: in case of **Toggle A ON** (Explicit Salt), in the ideal world, the functionality explicitly checks that the same $(\mathsf{msg}, \mathsf{salt})$ pair is not signed multiple times. However, this is not verified in the real world. Such an abort happens with negligible probability.

We thus consider the same sequence of hybrids as in [CEN25, Theorem 2] proofs, with an additional hybrid in which the simulator $\mathcal{S}$ no longer keeps track of the salt used in previous sessions. The statistical distance between the hybrids is thus the distance between sampling

with and without replacement from the set of possible salts. By a classical argument (see [Sta78]), the distance is $1 - \frac{1}{2^{\ell_s}} \cdot \prod_{i=0}^{q_s-1} \left(2^{\ell_s} - i\right) \approx \frac{q_s^2}{2^{\ell_s+1}}$ where q_s is the number of signing sessions and $\ell_s = \lambda + 64$ the bit length of the salt. Using the same parameters as in [BCC+23], this term is negligible, see [BCC+23, Section 5.3] for more details.Combining this with the other hybrids from the proof of [CEN25, Theorem 2], we conclude that Π_{ThrSign} statistically UC-realizes the protocol $\mathcal{F}_{\mathsf{ThrSign}}(\mathcal{L}, L)$ in the $\mathcal{F}_{\mathsf{Solve}}(L)$-hybrid model. □

5 Performance Evaluation

Table 3: Execution schedule of π_{Preproc} combined with Π_{Solve}, illustrating the order of local and online computations, and opportunities for parallelism. Colors indicate the effect of toggles: Toggle A enabled, Toggle B disabled, and Toggle B enabled.

	π_{Preproc}	Π_{Solve}
Local	Item 1: sample $[\![\mathbf{V}]\!]$	Item 1: sample $[\![\mathbf{R}]\!]$, $[\![\mathbf{S}]\!]$
online	Item 2: compute $[\![\mathbf{M}_i]\!]$	compute $[\![\mathbf{S}']\!] \leftarrow [\![\mathbf{U}]\!] \cdot [\![\mathbf{S}]\!]$
Local	Item 2: compute $[\![\mathbf{M}_i]\!]$, Item 3: compute $[\![\mathbf{A}]\!]$	
online	Item 4: compute $[\![\mathbf{Y}_i]\!]$	Item 2: compute $[\![\mathbf{R} \cdot \mathbf{A}]\!]$
Local	Item 4: compute $[\![\mathbf{y}]\!]$	
online	Item 5: generate salt via coin	Item 3, 4: compute $[\![\mathbf{R} \cdot \mathbf{A} \cdot \mathbf{S}]\!]$, $[\![-\mathbf{R} \cdot \mathbf{y}]\!]$
online		Item 5: open $[\![\mathbf{T}]\!]$
	Item 6: if $\mathbf{T}$ is full rank, continue; otherwise restart with fresh $[\![\mathbf{V}]\!]$, $[\![\mathbf{R}]\!]$, and $[\![\mathbf{S}]\!]$	
Local		Item 6, 7, 8: compute $\mathbf{T}^{-1}$, $[\![\mathbf{z}]\!]$, $[\![\mathbf{T}^{-1} \cdot \mathbf{h} + \mathbf{z}]\!]$, $[\![\mathbf{T}^{-1} \cdot \mathbf{R}]\!]$
online	Item 7: compute $[\![\mathbf{x}'_y]\!]$, $[\![\mathbf{B}]\!]$ using $[\![\mathbf{S}']\!]$	Item 10, 11: compute $[\![\mathbf{x}]\!]$, $[\![\mathbf{A}^{-1}]\!]$

In this section, we evaluate the practical performance of our optimizations by benchmarking three protocol variants. *Standard* corresponds to the baseline algorithm presented in [CEN25], but replaces the original solver with our modified Π_{Solve}. *Explicit-Standard* corresponds to our threshold protocol Π_{ThrSign} with **Toggle A** enabled. Finally, *Explicit-Depth-Reduced* instantiates Π_{ThrSign} with both **Toggle A** and **Toggle B** enabled. For each variant, we report results for both the non-active and actively secure versions, distinguishing between offline and online signing time. We do not provide benchmarks for the variants with **Toggle A** disabled, since it is clear that they will have worse online performance than if **Toggle A** is enabled: the advantage of disabling **Toggle A** lies in a security guarantee, not in performance. Note also that since we only performed local benchmarking it would not be meaningful to measure differences due only to latency reduction.

Remark 8. For a fair comparison, even though the original protocol in [CEN25] does not distinguish between offline and online phases, we measure its online time only as the time taken to perform message dependent computations, i.e., from Line 17 of Figure 2 onward. Further, we use the optimized Π_{Solve} procedure (without the solve-inv flag), since it gives a network improvement over the original solver, exchanging one matrix-to-matrix multiplication for one matrix-to-vector multiplication.

All experiments were conducted on macOS 15.6.1 (Darwin 24.6.0) running on Apple Silicon (ARM64), with an Apple M3 CPU (8 cores: 4 performance and 4 efficiency) and 24 GB of RAM. All `Go` benchmarks were compiled and executed using `Go 1.24.2` on `darwin/arm64`, and correspond to wall-clock time measured using Go's standard timing facilities. The reported results are the mean over 100 local signing executions that completed successfully, where each party performs the protocol rounds sequentially. The Go runtime was configured with `GOMAXPROCS=1`, restricting execution to a single logical core.

Complexity Analysis of Π_{ThrSign}. We first provide a coarse-grained analysis of the round complexity of the protocol Π_{ThrSign} when instantiated with the concrete solver Π_{Solve}.

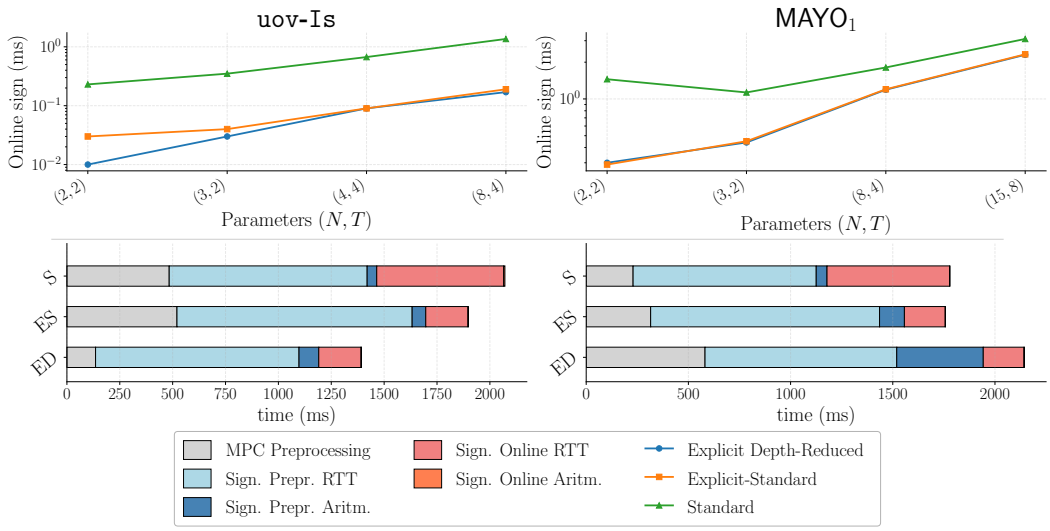

Figure 3: Online (top) and total (bottom) mean non-active signing times at security level 1, for selected parameter choices (N, T): left: `uov-Is`; right: MAYO$_1$. Top: shows varying (N, T); Bottom: uses $(N, T) = (2, 2)$, assuming 200ms round-trip time, where 'S' stands for 'Standard', 'ES' for 'Explicit-Standard' and 'ED' for 'Explicit-Depth-Reduced'.

Table 3 summarizes the order in which the different computations are performed and highlights the opportunities for parallel execution. It also makes explicit the differences induced by enabling Toggle A and Toggle B in π_{Preproc} (Procedure 3). We observe:

1. The dominant constraint on the number of online rounds is the multiplication depth of $\mathbf{T}$: it is 3 when Toggle B is disabled and 2 when Toggle B is enabled. In both cases, one additional round is required to open $\mathbf{T}$.

2. When Toggle B is disabled, the computations of $[\![\mathbf{Y}_i]\!]$ and $[\![-\mathbf{Ry}]\!]$ could be initiated earlier. However, this reordering does not reduce the overall number of rounds.

3. Enabling Toggle A requires two additional online rounds to generate salt via an interactive coin-tossing protocol, which can be executed in parallel with other online operations.

Let p_{inv} denote the probability that the matrix $\mathbf{T}$ is full rank. The expected round complexity of the preprocessing phase π_{Preproc} combined with Π_{Solve} is $1 + 4/p_{\mathsf{inv}}$ when Toggle A is disabled, and $1 + 3/p_{\mathsf{inv}}$ when Toggle A is enabled. Together with Table 2, this fully characterizes the round complexity of all protocol variants. If p_{inv} is the probability for the matrix $\mathbf{T}$ to be full-rank, then the expected round complexity of the preprocessing π_{Preproc} combined with Π_{Solve} is $1 + 4/p_{\mathsf{inv}}$ without Toggle A, and $1 + 3/p_{\mathsf{inv}}$ with Toggle A.

5.1 Emulation and Methodology

To validate the efficiency of our proposed optimizations and provide a reference for future deployments, we developed a complete emulation of our threshold protocol in `Golang`. The codebase[2] builds upon the existing threshold MAYO implementation at [NJKC25] but significantly re-engineers the core signing logic to integrate our optimized $\mathcal{F}_{\mathsf{ABB+solve}}$ procedure and the sharing strategies enabled by Toggles A and B.

[2] https://github.com/pqcvinaigrette/vinaigrette-go

Table 4: Signing performance for threshold MAYO over 100 samples. We report online signing time (and total signing time in parenthesis) in ms, comm./party and memory footprint is in kilobytes. Bold values indicate the best performance for each row and security model.

	(N,T)	**Standard** (3 online rounds) Non-Active	**Standard** (3 online rounds) Active	**Explicit-Salt** Non-Active	**Explicit-Salt** Active	**Explicit-Salt & Depth-Reduced** Non-Active	**Explicit-Salt & Depth-Reduced** Active
MAYO$_1$	(2,2)	1.45 (**252.38**)	5.73 (**1033.36**)	**0.29** (173.52)	4.99 (2007.09)	0.30 (555.42)	**4.82** (5182.61)
	(3,2)	1.13 (329.96)	9.98 (5190.42)	0.45 (**276.18**)	7.67 (**6591.91**)	**0.44** (790.84)	**7.34** (13798.31)
	(8,4)	1.81 (**458.39**)	23.20 (**11658.59**)	1.20 (656.26)	19.69 (18649.15)	**1.19** (1869.27)	**19.58** (38854.61)
	(15,8)	3.10 (1340.48)	48.12 (**27422.00**)	2.32 (**1332.61**)	37.25 (43788.12)	**2.30** (3415.58)	**37.02** (86646.07)
	Comm./party	4.0 (**138.4**)	4.0 (**141.0**)	**0.4** (213.4)	**0.4** (216.0)	**0.4** (1277.5)	**0.4** (1280.2)
	Mem. footprint	13.1	123.4	67.2	1141.0	33.6	570.5
MAYO 2	(2,2)	0.59 (**107.35**)	4.28 (**987.76**)	0.10 (135.84)	**1.57** (1411.28)	**0.09** (141.75)	1.58 (2086.99)
	(3,2)	0.78 (157.14)	7.56 (**1995.27**)	**0.15** (**104.09**)	2.46 (2780.42)	**0.15** (215.52)	**2.43** (3876.99)
	(8,4)	1.27 (310.54)	18.79 (**6585.45**)	0.39 (**241.41**)	6.69 (8021.24)	**0.38** (476.68)	**6.29** (11022.41)
	(15,8)	2.30 (**625.45**)	37.53 (**13911.18**)	**0.64** (885.40)	12.46 (18831.63)	**0.64** (1307.84)	**12.04** (25093.83)
	Comm./party	3.1 (**75.5**)	3.1 (**77.7**)	**0.2** (101.9)	**0.2** (104.1)	**0.2** (148.3)	**0.2** (149.5)
	Mem. footprint	8.9	81.5	20.9	354.0	10.5	177.0
MAYO 3	(2,2)	2.09 (**446.44**)	10.59 (**4612.44**)	**0.59** (657.28)	**9.72** (8080.74)	0.61 (1295.04)	9.66 (19020.37)
	(3,2)	2.52 (**706.44**)	17.93 (**9032.87**)	**0.81** (1192.01)	20.25 (15195.67)	0.83 (2931.95)	**14.85** (33738.64)
	(8,4)	3.80 (**959.69**)	42.10 (**25887.89**)	2.40 (1699.62)	39.63 (43157.55)	**2.37** (2340.86)	**39.21** (93693.50)
	(15,8)	7.19 (**1885.79**)	87.39 (**60560.82**)	**4.31** (2660.85)	**75.10** (97547.43)	4.33 (4476.15)	76.49 (207004.21)
	Comm./party	7.3 (**293.8**)	7.3 (**297.3**)	**0.6** (448.4)	**0.7** (452.0)	**0.6** (1478.3)	**0.7** (1480.1)
	Mem. footprint	24.5	229.5	139.5	2368.7	69.7	1184.3
MAYO 5	(2,2)	4.42 (**887.83**)	18.42 (**9563.09**)	1.07 (1260.80)	18.04 (17158.15)	**1.05** (2681.46)	**18.01** (42518.90)
	(3,2)	4.95 (1426.73)	30.52 (18171.56)	1.68 (**1349.10**)	27.43 (31164.23)	**1.55** (1435.96)	**27.41** (**109167.34**)
	(8,4)	7.87 (**2912.80**)	**72.23** (**51838.57**)	4.41 (2934.10)	72.77 (88909.45)	**4.22** (4753.47)	73.97 (202928.60)
	(15,8)	11.13 (5719.68)	152.12 (**119233.15**)	8.09 (**5061.08**)	141.51 (198541.92)	**7.97** (8878.01)	**140.92** (440112.29)
	Comm./party	12.3 (**557.6**)	12.3 (**562.2**)	**0.9** (844.1)	**0.9** (844.8)	**0.9** (5513.9)	**0.9** (5518.7)
	Mem. footprint	41.7	389.0	259.9	4415.6	129.9	2207.8

Our primary goal was to isolate the algorithmic performance—specifically the impact of reduced multiplicative depth and non-interactive preprocessing—from the variability of network conditions. By emulating the interactions of multiple parties within a single execution thread, we obtain deterministic and reproducible benchmarks that accurately reflect the computational overhead of the threshold logic itself. Running benchmark samples in parallel would reduce wall-clock time but distort per-operation measurements by introducing resource contention (CPU cache, memory bandwidth). The sequential design also mirrors real-world deployment, where parties run on separate machines and synchronize at each round boundary. The relevant performance metrics are therefore round complexity and communication cost, both of which are reported in the benchmarks. However, as noted in Table 3, there are opportunities for parallelism across operations, which we will explore as future work. We verified the correctness of the protocol by running the parties' code locally, simulating the exchange of messages via function calls rather than network sockets. This modular structure allows us to benchmark each optimization independently while keeping the surrounding protocol unchanged. We measured the memory footprint of each variant, defined as the memory allocated during preprocessing for the online phase.

We note that while our emulation is functionally complete for the purpose of verifying correctness and benchmarking computational costs, it does not include a network layer or the generation of correlated randomness (as discussed below). Our optimizations are purely algorithmic: we did not apply low-level software tuning (e.g. assembly optimizations[3]) or hardware-specific accelerations. For example, we did not enforce constant-time execution or perform explicit memory-access pattern checks, though we expect standard countermeasures

[3]We have begun exploring low-level optimizations, including hand-written assembly and the use of architecture-specific instructions (`CLMUL in x64, PMULL` in ARMv8).

to apply without changing the protocol structure. Another possible direction to further reduce the depth of the $\mathcal{F}_{\mathsf{ABB+solve}}$ function is to use randomized encodings of multiplication triples, as proposed in [RRK+24]. We leave this exploration to future work.

Active security and MACs. In the actively secure variants, we augment all shared values with information-theoretic message authentication codes (MACs) in the standard SPDZ-style manner. Concretely, each secret-shared value x is accompanied by a MAC $\gamma_x = \alpha \cdot x$, where α is a global secret key shared among the parties. All local computations are performed consistently on both values and MACs, while openings are verified by checking MAC correctness. Any deviation is detected except with negligible probability. This adds a constant-factor overhead in both computation and communication, which we explicitly account for in our benchmarks by reporting active and non-active variants separately.

Correlated randomness. We emphasize that our current emulation and benchmarks do not include the active generation of correlated randomness (e.g., multiplication triples and associated MACs), but instead we assume a trusted dealer, and leave the efficient distribution of this step as a future question. This is a common practice in the literature on secure multi-party computation, as the generation of such correlated randomness can be implemented via a variety of techniques orthogonal to the core protocol logic. Furthermore, how to best implement this generation remains an open question. One can either perform it via a distributed protocol (e.g. relying on somewhat homomorphic encryption [DPSZ12], oblivious transfer [KOS16] or pseudorandom correlation generators (PCGs) [BCGI18, BCG+19]), or rely on a trusted initialization, or even have a dedicated party provide this correlated randomness, as done for instance in Trilithium [DKLS25]. The performance figures provided here focus on the threshold signing protocol itself, assuming such randomness is available. We note that, for each signature, we precompute enough correlated randomness to repeat the for loop in Line 9 of Figure 2 up to 4 times, which for NIST-I parameters is sufficient to ensure a small probability of failure below 2^{-11}.

5.2 Evaluation

We evaluated the protocol for 3 variants for both active and non-active. We provide the full results for both MAYO and uov (as our techniques are easily extended for this scheme) in Table 4 and in Table 5, respectively. Across all security levels and schemes, *Explicit-Standard* and *Explicit-Depth-Reduced* consistently reduces online signing costs, often at some expense of offline computation. This trade-off is particularly attractive in latency-sensitive deployments, where preprocessing can be amortized. Figure 3 provides a visual breakdown of signing costs at security level 1 under non-active security. The top part shows the online costs; the lower part shows total signing time, assuming a round-trip time of 200ms[4]. Costs are broken down into three components: correlated randomness preprocessing, independent of the secret key (grey); signature share preprocessing (blue); and online signing (red). For the latter two, solid colors represent arithmetic operations and lighter colors represent communication overhead.

A primary motivation for our optimizations is to minimize the latency of the signing protocol, in particular for the message-dependent signing phase, which is often the critical bottleneck in practical threshold deployments. In such settings, offline preprocessing can be amortized or executed ahead of time, while the online phase lies on the critical path of user-facing and coordination operations. One thing that is not fully captured by our benchmarks

[4]Under RFC 6928 [CDCM13], a 4KB payload (roughly, $\approx$ 3 TCP segments of maximum size) will easily fit within the initial congestion window and can be transmitted in a single burst without intermediate acknowledgements. Empirical RTT measurements between AWS ap-northeast-2 (Seoul) and us-east-1 (Northern Virginia) consistently fall in the $\approx$180–220 ms range [aws26], with 200ms as a representative median for latency.

Table 5: Signing performance for threshold UOV over 100 samples. We report online signing time (and total signing time in parenthesis) in ms, comm./party and memory footprint is in kilobytes. Bold values indicate the best performance for each row and security model.

		Standard (3 online rounds)		**Explicit-Standard**		**Explicit Depth-Reduced**	
	(N,T)	**Non-Active**	**Active**	**Non-Active**	**Active**	**Non-Active**	**Active**
UOVIP	(2,2)	0.23 (113.93)	1.63 (368.45)	0.03 (125.15)	**0.03** (345.58)	**0.01** (**55.61**)	**0.03** (**265.35**)
	(3,2)	0.35 (133.40)	2.86 (962.68)	0.04 (148.86)	**0.04** (1080.16)	**0.03** (**67.47**)	**0.04** (**464.96**)
	(8,4)	0.67 (260.58)	6.6 (2769.89)	**0.09** (294.28)	0.90 (3071.16)	**0.09** (**142.49**)	**0.59** (**1288.83**)
	(15,8)	1.36 (533.09)	14.49 (6790.40)	0.19 (599.93)	1.74 (7469.75)	**0.17** (**279.80**)	**1.73** (**2959.77**)
	Comm./party	5.1 (157.4)	5.2 (159.0)	**0.1** (171.2)	**0.1** (172.8)	**0.1** (**60.1**)	**0.1** (**61.0**)
	Mem. footprint	7.8	39.5	10.1	89.9	5.1	45.0
UOVIS	(2,2)	0.60 (322.88)	8.20 (1650.47)	0.05 (351.51)	**0.05** (2369.03)	**0.04** (**130.21**)	**0.05** (**1574.88**)
	(3,2)	0.82 (382.58)	10.39 (4437.95)	**0.07** (420.89)	1.24 (5525.67)	**0.07** (**165.78**)	**1.20** (**1983.52**)
	(8,4)	1.47 (802.93)	35.26 (13420.89)	**0.18** (812.85)	3.35 (16191.90)	**0.18** (**485.02**)	**3.19** (**3346.92**)
	(15,8)	2.90 (1490.76)	56.07 (39492.02)	0.35 (1654.78)	6.45 (**33138.10**)	**0.34** (**673.41**)	**6.34** (35138.70)
	Comm./party	5.2 (223.5)	5.3 (225.6)	**0.1** (237.8)	**0.1** (240.0)	**0.1** (**60.1**)	**0.1** (**114.2**)
	Mem. footprint	8.2	74.2	10.4	174.4	5.2	87.2
UOVIII	(2,2)	0.85 (465.41)	5.41 (2985.13)	**0.03** (517.49)	**0.06** (1332.11)	**0.03** (**129.54**)	**0.06** (**1104.01**)
	(3,2)	1.10 (558.35)	8.29 (3987.10)	0.09 (609.50)	**0.81** (4332.11)	**0.09** (**228.37**)	**0.81** (**1488.71**)
	(8,4)	1.95 (1066.59)	17.29 (11269.96)	0.23 (1186.85)	2.20 (12267.57)	**0.22** (**485.39**)	**2.08** (**4069.52**)
	(15,8)	3.82 (2178.69)	38.41 (27763.51)	0.44 (2406.27)	4.59 (29967.49)	**0.43** (**937.23**)	**4.05** (**9110.88**)
	Comm./party	13.6 (649.6)	13.6 (652.0)	**0.2** (686.5)	**0.2** (688.9)	**0.2** (**161.5**)	**0.2** (**162.8**)
	Mem. footprint	20.6	103.9	26.6	238.2	13.3	119.1
UOVV	(2,2)	1.64 (1068.28)	9.67 (5658.13)	**0.05** (937.22)	0.79 (5931.55)	0.10 (**376.36**)	**0.74** (**2993.44**)
	(3,2)	2.49 (1267.99)	13.30 (9086.45)	0.15 (1382.38)	1.39 (9832.58)	**0.15** (**474.48**)	**1.34** (**2991.14**)
	(8,4)	3.64 (2426.39)	29.97 (25868.95)	0.39 (2691.58)	3.74 (28092.88)	**0.38** (**1020.15**)	**3.65** (8214.29)
	(15,8)	6.92 (4969.63)	65.94 (64228.58)	0.76 (5464.14)	7.63 (68121.34)	**0.74** (**1932.85**)	**6.82** (**17953.47**)
	Comm./party	11.9 (743.0)	12.0 (744.6)	**0.1** (775.6)	**0.1** (777.2)	**0.1** (**142.6**)	**0.1** (**143.4**)
	Mem. footprint	36.4	183.5	46.6	418.8	23.3	209.4

is the impact of the number of interaction rounds on overall latency. In fact, network latency is a primary contributor to end-to-end latency, particularly in wide-area or geo-distributed settings. Extensive work in distributed systems and networking [DB13, CDE+13, Kle17], has shown that reducing the number of synchronous coordination points is often more impactful than optimizing local computation, since round-trip delays dominate overall latency, specially if they depend on coordination.

Comparison to original proposal [CEN25]. Across all security levels and parameter choices, our results show that our new protocols have the potential of consistently reducing online latency. Our benchmarks confirm that the additional offline cost incurred by our modification is manageable, remaining within a small constant factor of the baseline across all settings.

The impact of network costs is visualized in Figure 3. From it, we see that, for the online signing phase, network costs are the main contributor to the overall latency, and that our optimizations significantly reduce them with respect to the (optimized) standard protocol, marking a clear difference from the results in [CEN25].

We emphasize that not all deployment scenarios allow for extensive preprocessing. In highly constrained or on-demand settings, where signatures are produced sporadically and preprocessing cannot be amortized, the **Standard** and **Explicit-Standard** variants offer meaningful latency improvements for several parameter choices, thanks to the smaller overall number of operations required. For example, for MAYO_1 with $(N, T) = (2, 2)$, the results from Figure 3 show that the end-to-end signing time is minimized by the **Standard** variant.

Trade-offs for Depth Reduction variant. The **Explicit-Depth-Reduced** variant always reduces the average number of offline rounds with respect to the other variants, but incurs in additional arithmetic overhead, due to the larger linear system (Line 15 of Figure 2). Thus, the latency improvement of this variant depends on the relative cost of the computational cost of the protocol with respect to the round-trip time of the interactions.

This arithmetic overhead mainly depends on the parameter k, which defines the system expansion ratio for the MAYO scheme and is implicitly set to 1 in `uov`. Thus we expect that the **Depth-Reduced** is more beneficial for `uov` parameter sets ($k = 1$) and MAYO_2 ($k = 4$), while for other MAYO parameter sets ($k \geq 10$) the additional arithmetic cost may outweigh the benefits of reducing the number of rounds. The results in Figure 3 confirm this intuition for the non-active setting, comparing `uov-Is` and MAYO_1. Note that due to the oil space adjustment (Item 7 of π_{Preproc}) the parameter k also effects the arithmetic cost of the **Explicit-Standard** variant as well, explaining why for MAYO_1 the **Standard** variant outperforms the new variants.

References

[ADEO21] Diego F. Aranha, Anders P. K. Dalskov, Daniel Escudero, and Claudio Orlandi. Improved threshold signatures, proactive secret sharing, and input certification from LSS isomorphisms. In Patrick Longa and Carla Ràfols, editors, *LATINCRYPT 2021*, volume 12912 of *LNCS*, pages 382–404. Springer, Cham, October 2021.

[ADN06] Jesús F. Almansa, Ivan Damgård, and Jesper Buus Nielsen. Simplified threshold RSA with adaptive and proactive security. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 593–611. Springer, Berlin, Heidelberg, May / June 2006.

[aws26] AWS latency test. https://aws-latency-test.com/, 2026. Accessed: March 2026.

[BCC+23] Ward Beullens, Fabio Campos, Sofía Celi, Basil Hess, and Matthias J. Kannwischer. MAYO. Technical report, National Institute of Standards and Technology, 2023. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[BCC+24] Ward Beullens, Fabio Campos, Sofía Celi, Basil Hess, and Matthias J. Kannwischer. MAYO. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[BCD+24] Ward Beullens, Ming-Shing Chen, Jintai Ding, Boru Gong, Matthias J. Kannwischer, Jacques Patarin, Bo-Yuan Peng, Dieter Schmidt, Cheng-Jhih Shih, Chengdong Tao, and Bo-Yin Yang. UOV — Unbalanced Oil and Vinegar. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[BCG+19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 291–308. ACM Press, November 2019.

[BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 896–912. ACM Press, October 2018.

[BCH+23a] Ward Beullens, Ming-Shing Chen, Shih-Hao Hung, Matthias J. Kannwischer, Bo-Yuan Peng, Cheng-Jhih Shih, and Bo-Yin Yang. Oil and vinegar: Modern parameters and implementations. Cryptology ePrint Archive, Report 2023/059, 2023.

[BCH+23b] Ward Beullens, Ming-Shing Chen, Shih-Hao Hung, Matthias J. Kannwischer, Bo-Yuan Peng, Cheng-Jhih Shih, and Bo-Yin Yang. Oil and vinegar: Modern parameters and implementations. *IACR TCHES*, 2023(3):321–365, 2023.

[BdCE+25] Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Efficient, scalable threshold ML-DSA signatures: An MPC approach. Cryptology ePrint Archive, Paper 2025/1163, 2025.

[Beu22] Ward Beullens. MAYO: Practical post-quantum signatures from oil-and-vinegar maps. In Riham AlTawy and Andreas Hülsing, editors, *SAC 2021*, volume 13203 of *LNCS*, pages 355–376. Springer, Cham, September / October 2022.

[BKL+25] Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical two-round threshold signatures from learning with errors. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 149–164. IEEE Computer Society Press, May 2025.

[BKP13] Rikke Bendlin, Sara Krehbiel, and Chris Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In Michael J. Jacobson, Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh Safavi-Naini, editors, *ACNS 2013*, volume 7954 of *LNCS*, pages 218–236. Springer, Berlin, Heidelberg, June 2013.

[BMP22] Constantin Blokh, Nikolaos Makriyannis, and Udi Peled. Efficient asymmetric threshold ECDSA for MPC-based cold storage. Cryptology ePrint Archive, Paper 2022/1296, 2022.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.

[CCL+19] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Two-party ECDSA from hash proof systems and efficient instantiations. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 191–221. Springer, Cham, August 2019.

[CDCM13] Jerry Chu, Nandita Dukkipati, Yuchung Cheng, and Matt Mathis. Increasing TCP's Initial Window, April 2013. Experimental.

[CDE+13] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. Spanner: Google's globally distributed database. *ACM Trans. Comput. Syst.*, 31(3), August 2013.

[CdPE+26] Sofía Celi, Rafaël del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Efficient threshold ML-DSA. Cryptology ePrint Archive, Paper 2026/013, 2026.

[CEN25] Sofía Celi, Daniel Escudero, and Guilhem Niot. Share the MAYO: Thresholdizing MAYO. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *Post-Quantum Cryptography - 16th International Workshop, PQCrypto 2025, Part I*, pages 165–198. Springer, Cham, April 2025.

[CGG+20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1769–1787. ACM Press, November 2020.

[CKM23] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023.

[CLOS02] Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai. Universally composable two-party and multi-party secure computation. Cryptology ePrint Archive, Report 2002/140, 2002.

[Cor00] Jean-Sébastien Coron. On the exact security of full domain hash. In Mihir Bellare, editor, *CRYPTO 2000*, volume 1880 of *LNCS*, pages 229–235. Springer, Berlin, Heidelberg, August 2000.

[CS19] Daniele Cozzo and Nigel P. Smart. Sharing the LUOV: Threshold post-quantum signatures. In Martin Albrecht, editor, *17th IMA International Conference on Cryptography and Coding*, volume 11929 of *LNCS*, pages 128–153. Springer, Cham, December 2019.

[DB13] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Commun. ACM*, 56(2):74–80, February 2013.

[DKLs18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Secure two-party threshold ECDSA from ECDSA assumptions. In *2018 IEEE Symposium on Security and Privacy*, pages 980–997. IEEE Computer Society Press, May 2018.

[DKLS25] Antonín Dufka, Semjon Kravtšenko, Peeter Laud, and Nikita Snetkov. Trilithium: Efficient and universally composable distributed ML-DSA signing. Cryptology ePrint Archive, Report 2025/675, 2025.

[DKM+24] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.

[DM20] Luca De Feo and Michael Meyer. Threshold schemes from isogeny assumptions. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 187–212. Springer, Cham, May 2020.

[DN03] Ivan Damgård and Jesper Buus Nielsen. Universally composable efficient multiparty computation from threshold homomorphic encryption. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 247–264. Springer, Berlin, Heidelberg, August 2003.

[DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012.

[EKT24] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024.

[GG19] Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ECDSA with fast trustless setup. Cryptology ePrint Archive, Report 2019/114, 2019.

[GHKR08] Rosario Gennaro, Shai Halevi, Hugo Krawczyk, and Tal Rabin. Threshold RSA for dynamic and ad-hoc groups. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 88–107. Springer, Berlin, Heidelberg, April 2008.

[KCLM22] Irakliy Khaburzaniya, Konstantinos Chalkias, Kevin Lewi, and Harjasleen Malvai. Aggregating and thresholdizing hash-based signatures using STARKs. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 393–407. ACM Press, May / June 2022.

[KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.

[Kle17] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.

[KMOS21] Yashvanth Kondi, Bernardo Magri, Claudio Orlandi, and Omer Shlomovits. Refresh when you wake up: Proactive threshold wallets with offline devices. In *2021 IEEE Symposium on Security and Privacy*, pages 608–625. IEEE Computer Society Press, May 2021.

[KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 830–842. ACM Press, October 2016.

[KPG99] Aviad Kipnis, Jacques Patarin, and Louis Goubin. Unbalanced Oil and Vinegar signature schemes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 206–222. Springer, Berlin, Heidelberg, May 1999.

[LDK+22] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

[LNR18] Yehuda Lindell, Ariel Nof, and Samuel Ranellucci. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. Cryptology ePrint Archive, Report 2018/987, 2018.

[Mak22] Nikolaos Makriyannis. On the classic protocol for MPC schnorr signatures. Cryptology ePrint Archive, Paper 2022/1332, 2022.

[Nat] National Institute of Standards and Technology (NIST). Multiparty threshold cryptography. https://csrc.nist.gov/projects/threshold-cryptography. Accessed: 2025-01-04.

[NJKC25] Andreas Skriver Nielsen, Markus Valdemar Grønkjær Jensen, Hans-Christian Kjeldsen, and Sofía Celi. mayo-threshold-go: Threshold MAYO in Go. https://github.com/AU-HC/mayo-threshold-go, 2025. Accessed: 2026-01-16.

[RRK+24] Pascal Reisert, Marc Rivinius, Toomas Krips, Sebastian Hasler, and Ralf Küsters. Actively secure polynomial evaluation from shared polynomial encodings. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part VI*, volume 15489 of *LNCS*, pages 3–35. Springer, Singapore, December 2024.

[Sta78] Adriaan Johannes Stam. Distance between sampling with and without replacement. *Statistica Neerlandica*, 32(2):81–91, 1978.